

Consolidated Data Mining Notes

Consolidated Data Mining Notes

Formatted from Consolidated_Data_Mining_Notes.md

Reference format: IS ZC415_SS ZC425_Data_Mining -
Watermarked.pdf

Content source: PDF extraction, transcripts, and
recordings listed in the notes

1

Quick Index (1/2)

Session | Main coverage | PDF refs
CS 1 | Data mining introduction, KDD, major tasks, applications,
EDA orientation | pp. 1-14, 356-406
CS 2 | Data objects, attributes, data quality,
similarity/distance, preprocessing, feature selection | pp.
15-58
CS 3 | Classification basics and decision trees | pp. 59-88
CS 4 | Decision tree split criteria, overfitting, pruning,
validation, ANN intro | pp. 73-114
CS 5 | Neural networks, backpropagation, ensemble methods, class
imbalance intro | pp. 104-142
CS 6 | Imbalanced classification metrics, ROC/AUC, KNN, rule
classifiers, association rules intro | pp. 129-184
CS 7 | Frequent itemsets, interestingness measures, support skew,
sequential/subgraph mining, clustering intro | pp. 189-260
CS 8 | Clustering: k-means, hierarchical, DBSCAN, validity,
advanced clustering | pp. 261-309, 885-929
CS 9 | Anomaly detection, web/data mining trends, social/big-data
analytics, false discovery/statistical testing | pp. 310-422,
930-946
CS 10 | Web intelligence, PageRank, recommender systems,
collaborative filtering | pp. 423-561
CS 11 | Recommendation variants, text mining, topic modeling, fake

2

Quick Index (2/2)

news, LSA/LDA | pp. 562-619
CS 12 | Data stream mining, concept drift, one-pass constraints,
sampling/sketching/synopsis | pp. 620-649
CS 13 | Cybercrime/digital forensics case study, graph streams and
dynamic networks | pp. 650-757
CS 14 | Graph pattern mining, graph anomalies, graph generators,
partitioning, cross-association, multimedia/DSP appendix topics
| pp. 686-884

3

CS 1 - Introduction to Data Mining and KDD

CS 1 - Introduction to Data Mining and KDD

Session divider

4

CS 1 - Introduction to Data Mining and KDD: Why Data Mining Is Needed

Data mining is needed because modern organizations collect data at a scale where manual analysis is not enough. Commercial sources include web logs, e-commerce visits, banking and credit-card transactions, customer relationship data, retail transactions, and social-network activity. Scientific sources include satellite sensors, sky surveys, biological experiments, climate simulations, and other high-throughput instruments (PDF pp. 1-3).

The purpose is not merely storage. The key idea is to turn large-scale raw data into useful patterns, predictions, and decisions. Data mining becomes important when data is large, high-dimensional, heterogeneous, distributed, noisy, or too complex for traditional query/reporting methods (PDF p. 4).

5

CS 1 - Introduction to Data Mining and KDD: What Data Mining Means

Data mining is the non-trivial extraction of implicit, previously unknown, and potentially useful information from data. Another way to say it: it is automatic or semi-automatic exploration and analysis of large quantities of data to discover meaningful patterns (PDF p. 4).

Data mining draws from:

- Machine learning and artificial intelligence
- Pattern recognition
- Statistics
- Database systems
- Data warehousing and large-scale data management

It is a central component of data science and knowledge discovery. The broader KDD process includes selecting data, cleaning it, transforming it, mining patterns/models, evaluating them, and deploying/interpreting the discovered knowledge.

6

CS 1 - Introduction to Data Mining and KDD: Data Warehousing and OLAP Context

Data warehouses organize integrated historical data for reporting and analysis. OLAP supports multidimensional summaries such as sales by region, month, product, or customer segment. Data mining goes beyond OLAP summaries by automatically discovering models and patterns such as classifiers, clusters, association rules, anomalies, and trends.

Exam distinction: OLAP answers analyst-driven aggregation questions; data mining discovers predictive or descriptive patterns that may not be specified in advance.

7

CS 1 - Introduction to Data Mining and KDD: Data Mining Versus Related Areas

Data mining is not just database querying. A query answers a known question over stored data; data mining searches for patterns or models that may not be known in advance. It is also not identical to statistics, although statistics supplies probability, estimation, hypothesis testing, and uncertainty reasoning. Data mining focuses heavily on scalable algorithms, automated discovery, high-dimensional data, and real-world predictive/descriptive workflows.

8

CS 1 - Introduction to Data Mining and KDD: Prediction and Description

The course repeatedly divides tasks into two major types (PDF p. 5):

- Prediction methods: use known variables to predict unknown or future values. Classification and regression are typical examples.
- Description methods: find human-interpretable patterns that describe the data. Clustering, association rules, summarization, and anomaly/change detection are typical examples.

This distinction is useful in exams: if the target variable is known and the model learns to predict it, think predictive modeling. If there is no predefined target and the model searches for structure, think descriptive mining.

9

CS 1 - Introduction to Data Mining and KDD: Major Data Mining Tasks

Classification: Learn a model that maps input attributes to a predefined class label. Examples include fraud detection, churn prediction, tumor classification, news categorization, intrusion detection, and sky-object cataloging (PDF pp. 6-8).

Clustering: Group objects so that objects in the same group are more similar to each other than to objects in other groups. Clustering is useful for customer segmentation, document grouping, gene/protein grouping, stock grouping, and summarizing large data sets (PDF pp. 10-11).

Association rule discovery: Find dependency rules that predict the occurrence of one item from other items. The classic example is market-basket analysis, such as {Diaper, Milk} → {Beer} or {Milk} → {Coke} (PDF p. 12).

Anomaly/deviation/change detection: Detect significant deviations from normal behavior. Applications include credit-card fraud, network intrusion, sensor monitoring, surveillance, and detecting changes in global forest cover (PDF p. 13).

10

CS 1 - Introduction to Data Mining and KDD: Application View

The lectures emphasize that the same mining tasks appear in different domains. For example, classification can appear as fraud detection in banking, disease prediction in health care, or spam filtering in text. Association analysis can appear as market-basket mining, alarm diagnosis in telecom, or symptom/test association in medical informatics. Clustering can appear in market segmentation or document browsing. Anomaly detection can appear in finance, cybersecurity, sensor networks, and environmental monitoring (PDF pp. 7-13).

11

CS 1 - Introduction to Data Mining and KDD: Exploratory Data Analysis Orientation

Before applying mining algorithms, data must be explored. EDA is used to understand distributions, missing values, outliers, relationships among variables, data quality issues, and whether the chosen mining task makes sense. In later sessions, this becomes important for preprocessing, distance selection, class imbalance, clustering validity, anomaly detection, and stream mining.

12

CS 1 - Introduction to Data Mining and KDD: Exam Pointers

- Be able to distinguish predictive versus descriptive tasks.
- Be able to map examples to tasks: fraud detection is classification/anomaly detection depending on labels; market basket is association analysis; customer segmentation is clustering.
- Data mining is part of a pipeline: collect, clean, transform, mine, evaluate, and use.

13

CS 1 - Introduction to Data Mining and KDD: Formulae/Numericals Highlight

No heavy numerical formula is central in CS 1. The important "must remember" item is conceptual classification:

- Conceptual rule: predefined label available -> classification/regression; no label and grouping required -> clustering; co-occurring items -> association rules; rare/deviating behavior -> anomaly/change detection.

14

CS 2 - Data, Attributes, Similarity, and Preprocessing

CS 2 - Data, Attributes, Similarity, and Preprocessing

Session divider

15

CS 2 - Data, Attributes, Similarity, and Preprocessing: Data Objects and Attributes

A data set is made of data objects, also called records, points, samples, entities, or instances. Each object is described by attributes. An attribute is a property or characteristic of an object, such as age, income, color, transaction amount, or document term frequency (PDF pp. 15-20).

Attribute type determines what operations are meaningful:

- Nominal: categories with no order, such as color, gender, zip code.
- Binary: two-valued nominal attribute, such as yes/no or 0/1.
- Ordinal: ordered categories where differences are not necessarily numeric, such as low/medium/high.
- Interval: numeric values where differences are meaningful but ratios are not, such as temperature in Celsius.
- Ratio: numeric values where both differences and ratios are meaningful, such as weight, age, count, or income.

This matters because distance, similarity, normalization, and mining algorithm choices depend on attribute type.

16

CS 2 - Data, Attributes, Similarity, and Preprocessing: Types of Data Sets

The course touches several data forms:

- Record data: tables, data matrices, transaction data.
- Document/text data: documents represented by terms or word counts.
- Transaction/basket data: each record contains a set of items.
- Graph/network data: nodes and edges.
- Ordered/sequential data: time series, sequences, data streams.
- Spatial, image, multimedia, and web data.

Different data forms require different representations. For example, transaction data is natural for association rules, text data needs vectorization/topic models, graph data needs graph mining methods, and streams need one-pass algorithms.

17

CS 2 - Data, Attributes, Similarity, and Preprocessing: Data Quality

Data mining quality is limited by data quality. Major problems include (PDF pp. 21-25):

- Noise: random error or variance in measured attributes.
- Outliers: objects with characteristics very different from most data.
- Missing values: absent attribute values due to collection or integration issues.
- Duplicate data: same entity represented more than once.
- Inconsistent values: conflicts due to different sources, units, or naming conventions.

Handling data quality is not optional. Bad data can create bad patterns, misleading similarity values, poor classifiers, and false anomalies.

18

CS 2 - Data, Attributes, Similarity, and Preprocessing: Similarity and Dissimilarity

Similarity measures how alike two objects are. Dissimilarity or distance measures how different they are. Usually:

- Similarity is high when objects are alike.
- Distance/dissimilarity is low when objects are alike.
- Similarity often lies in $[0, 1]$, but not always.
- Distance is often non-negative and may satisfy metric properties.

Metric distance usually has these properties: non-negativity, identity, symmetry, and triangle inequality.

19

CS 2 - Data, Attributes, Similarity, and Preprocessing: Common Distance Measures

Use distance measures based on attribute type and data representation. Numeric vectors often use Euclidean, Manhattan, or Minkowski distance. Binary vectors may use simple matching or Jaccard. Text often uses cosine similarity after vectorization.

Formula - Euclidean distance (PDF p. 31):

```
'''text
d(x, y) = sqrt( sum_k (x_k - y_k)^2 )
'''
```

Use Euclidean distance for continuous numeric attributes when scale is meaningful and attributes are comparable.

Formula - Minkowski distance (PDF pp. 32-33):

```
'''text
d_r(x, y) = ( sum_k |x_k - y_k|^r )^(1/r)
'''
```

Special cases:

- $r = 1$: Manhattan / city-block distance.
- $r = 2$: Euclidean distance.
- $r \rightarrow \infty$: supremum / maximum-coordinate distance.

20

CS 2 - Data, Attributes, Similarity, and Preprocessing: Binary Similarity

For binary attributes, count:

- f11: number of attributes where both objects are 1.
- f00: number of attributes where both are 0.
- f10: object x is 1 and y is 0.
- f01: object x is 0 and y is 1.

Formula - Simple Matching Coefficient (SMC) (PDF p. 36):

```
```text
SMC = (f11 + f00) / (f01 + f10 + f11 + f00)
```
```

SMC treats co-absence (0,0) as important.

Formula - Jaccard coefficient (PDF p. 36):

```
```text
Jaccard = f11 / (f01 + f10 + f11)
```
```

Jaccard ignores co-absence. It is useful for asymmetric binary data such as market baskets, where "both did not buy item X" is usually not informative.

21

CS 2 - Data, Attributes, Similarity, and Preprocessing: Entropy and Information (1/2)

Entropy measures uncertainty or impurity of a probability distribution.

Formula - Entropy (PDF pp. 41-42):

```
```text
H(X) = - sum_i p_i log2(p_i)
```
```

Numerical: a fair coin has p(head)=0.5, p(tail)=0.5, so:

```
```text
H(X) = -0.5 log2(0.5) - 0.5 log2(0.5) = 1 bit
```
```

Low entropy means predictable; high entropy means uncertain. Entropy later reappears in decision-tree splitting and information gain.

Formula - Mutual information (PDF p. 43):

```
```text
I(X, Y) = H(X) + H(Y) - H(X, Y)
```
```

22

CS 2 - Data, Attributes, Similarity, and Preprocessing: Entropy and Information (2/2)

feature relevance, and association.

23

CS 2 - Data, Attributes, Similarity, and Preprocessing: Data Preprocessing

Preprocessing transforms raw data into a form suitable for mining.
Common operations include (PDF pp. 44-58):

- Aggregation: combine multiple records or attributes into summaries.
- Sampling: use a representative subset when the full data is too large.
- Dimensionality reduction: reduce number of attributes while preserving useful information.
- Feature subset selection: keep relevant features and remove redundant or irrelevant features.
- Feature creation: construct new attributes from existing attributes.
- Discretization and binarization: convert continuous attributes into intervals or binary attributes.
- Attribute transformation: normalize, standardize, log-transform, or otherwise scale attributes.

24

CS 2 - Data, Attributes, Similarity, and Preprocessing: Sampling

Sampling is useful when data is too large to process fully. A sample should preserve important distributional properties of the population. Random sampling avoids systematic bias, but rare classes and outliers can be missed if sampling is careless. Stratified sampling may be needed for imbalanced classes.

25

CS 2 - Data, Attributes, Similarity, and Preprocessing: Dimensionality Reduction and PCA

High-dimensional data can make distance measures less meaningful, increase computational cost, and cause overfitting. Dimensionality reduction reduces attributes while preserving information. PCA is a common method that transforms correlated variables into orthogonal principal components ordered by explained variance.

The exam point: PCA creates new transformed features; feature selection chooses a subset of existing features.

26

CS 2 - Data, Attributes, Similarity, and Preprocessing: Discretization and Binarization

Discretization converts continuous attributes into intervals, such as age into young/middle/old. Binarization converts categorical or set-valued information into binary indicator attributes. These are useful for rule mining, decision trees, and algorithms that expect categorical inputs.

27

CS 2 - Data, Attributes, Similarity, and Preprocessing: Exam Pointers

- Attribute type controls valid operations.
- Jaccard is preferred when 0,0 matches are not meaningful.
- Entropy is uncertainty; mutual information is dependence.
- Preprocessing is part of the mining pipeline, not an optional cleanup step.
- PCA and feature selection are different.

28

CS 2 - Data, Attributes, Similarity, and Preprocessing: Formulae/Numericals Highlight

- Formula: $d(x,y)=\sqrt{\sum_k (x_k-y_k)^2}$ for Euclidean distance (PDF p. 31).
- Formula: $d_r(x,y)=(\sum_k |x_k-y_k|^r)^{1/r}$ for Minkowski distance (PDF pp. 32-33).
- Formula: $SMC=(f_{11}+f_{00})/(f_{01}+f_{10}+f_{11}+f_{00})$ (PDF p. 36).
- Formula: $Jaccard=f_{11}/(f_{01}+f_{10}+f_{11})$ (PDF p. 36).
- Formula: $H(X)=-\sum_i p_i \log_2(p_i)$ (PDF pp. 41-42).
- Numerical: fair coin entropy is 1 bit (PDF pp. 41-42).
- Formula: $I(X,Y)=H(X)+H(Y)-H(X,Y)$ (PDF p. 43).

29

CS 3 - Classification and Decision Tree Basics

CS 3 - Classification and Decision Tree Basics

Session divider

30

CS 3 - Classification and Decision Tree Basics: Classification Problem

Classification is a supervised learning task. Given a training set of records, each record is represented as (x, y) , where x is the attribute set and y is the class label. The task is to learn a model that maps each attribute set x to one of the predefined class labels y (PDF p. 59).

Terminology:

- x : attributes, predictors, independent variables, inputs.
- y : class, response, dependent variable, output.
- Training set: labeled data used to build the model.
- Test set: separate data used to estimate performance.
- Model/classifier: learned mapping from attributes to class labels.

Examples include email spam classification, tumor diagnosis, galaxy classification, credit worthiness prediction, fraud detection, and intrusion detection (PDF pp. 60-61).

31

CS 3 - Classification and Decision Tree Basics: General Approach

The general classification workflow is:

1. Collect labeled training data.
2. Learn a classifier from the training data.
3. Apply the classifier to unseen records.
4. Evaluate using test data.

The key exam distinction: training data is used to fit the model; test data estimates generalization. Do not evaluate final model quality only on training data.

32

CS 3 - Classification and Decision Tree Basics: Classification Techniques

The PDF lists common base classifiers and ensemble classifiers (PDF p. 61):

- Decision trees
- Rule-based classifiers
- Nearest-neighbor classifiers
- Naive Bayes and Bayesian belief networks
- Support vector machines
- Neural networks and deep neural networks
- Ensemble methods: bagging, boosting, random forests

CS 3 focuses mainly on decision trees.

33

CS 3 - Classification and Decision Tree Basics: Decision Tree Representation

A decision tree is a tree-structured classifier:

- Internal node: test on an attribute.
- Branch: outcome of the test.
- Leaf node: class label or class distribution.

To classify a record, start at the root, follow branches according to attribute values, and end at a leaf. The leaf gives the predicted class.

Decision trees are popular because they are easy to interpret, fast at prediction, and can handle both categorical and continuous attributes with suitable split rules.

34

CS 3 - Classification and Decision Tree Basics: Example Decision Tree

The PDF borrower example uses attributes like home owner, marital status, and income to predict whether a borrower defaulted (PDF pp. 61-65). The tree may split first on Home Owner, then on Marital Status, then on Income. More than one decision tree can fit the same data, which creates the need for a criterion to choose good splits (PDF p. 65).

35

CS 3 - Classification and Decision Tree Basics: Hunt's Algorithm

Hunt's algorithm is the core recursive idea behind many decision-tree methods (PDF pp. 66-67):

1. If all records at a node belong to the same class, make the node a leaf with that class.
2. If records have multiple classes, select an attribute test that partitions records into smaller subsets.
3. Recursively apply the same procedure to each child node.

Decision-tree algorithms differ mainly in how they choose the split, how they handle stopping, and how they prune.

36

CS 3 - Classification and Decision Tree Basics: Split Selection

A good split should create child nodes that are purer than the parent node. Purity means records in a child mostly belong to one class. Impurity measures include Gini index, entropy, and classification error (PDF pp. 73-84).

Decision-tree induction is usually greedy: at each node, choose the locally best split according to an impurity or gain measure. This does not guarantee a globally optimal tree, but it is practical.

37

CS 3 - Classification and Decision Tree Basics: Attribute Test Conditions

Different attribute types require different tests (PDF pp. 68-72):

- Binary attributes: split into two branches.
- Nominal attributes: multiway split or binary partition of categories.
- Ordinal attributes: preserve order when splitting.
- Continuous attributes: choose threshold splits such as $\text{Income} < 80K$.

Continuous split selection often sorts values and evaluates candidate thresholds between consecutive values.

38

CS 3 - Classification and Decision Tree Basics: Decision Tree Algorithms

Common decision-tree algorithms include:

- ID3: uses information gain and handles categorical attributes.
- C4.5: extends ID3, handles continuous attributes, missing values, pruning, and gain ratio.
- CART: uses binary splits and commonly uses Gini index for classification.

The lecture uses these names to connect the general tree idea to well-known algorithms.

39

CS 3 - Classification and Decision Tree Basics: Advantages and Disadvantages

Advantages (PDF p. 86):

- Relatively inexpensive to construct.
- Very fast for classifying unknown records.
- Easy to interpret if the tree is small.
- Robust to noise if overfitting is controlled.
- Can handle redundant attributes.
- Can handle irrelevant attributes in many cases.

Disadvantages:

- Greedy splitting may miss interacting attributes.
- Each decision boundary usually involves a single attribute.
- Trees can overfit if grown too deep.
- Small data changes can sometimes change tree structure.

40

CS 3 - Classification and Decision Tree Basics: Exam Pointers

- Know how to classify a record by traversing a tree.
- Know why splitting criterion matters.
- Know the difference between root, internal node, branch, and leaf.
- Understand that a decision tree can perfectly fit training data and still generalize poorly.

41

CS 3 - Classification and Decision Tree Basics: Formulae/Numericals Highlight

CS 3 introduces the need for impurity measures but most formula-heavy split criteria are emphasized in CS 4.

- Numerical pattern: Given a table and candidate split, count class frequencies in each child node, compute impurity, then choose the split with lower weighted impurity or higher information gain.

42

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro

Session divider

43

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Gini Index

Gini measures node impurity. If all records at a node belong to one class, Gini is 0. If classes are mixed, Gini is higher.

Formula - Gini impurity (PDF pp. 73-76):

```
```text
GINI(t) = 1 - sum_i p_i(t)^2
```
```

where $p_i(t)$ is the fraction of records at node t that belong to class i .

For a split, compute weighted Gini over child nodes:

```
```text
GINI_split = sum_i (n_i / n) GINI(i)
```
```

where n_i is the number of records in child node i , and n is the number of records in the parent node.

Choose the split with the lowest weighted Gini.

44

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Entropy and Information Gain

Entropy is another impurity measure.

Formula - Entropy at node t (PDF pp. 78-82):

```
'''text
Entropy(t) = - sum_i p_i(t) log2 p_i(t)
'''
```

Information gain measures the reduction in entropy after a split:

```
'''text
Gain = Entropy(parent) - weighted_entropy(children)
'''
```

Choose the split with the highest information gain.

45

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Gain Ratio

Information gain can prefer attributes with many distinct values.
Gain ratio corrects this by dividing by split information (PDF pp. 82-83).

Formula - Gain Ratio:

```
'''text
Gain Ratio = Gain_split / SplitInfo
SplitInfo = - sum_i (n_i / n) log2(n_i / n)
'''
```

Use gain ratio when you want to reduce bias toward many-way splits.

46

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Classification Error

Classification error measures the fraction of records not belonging to the majority class at a node.

Formula - Classification error (PDF p. 83):

```
```text
Error(t) = 1 - max_i p_i(t)
```
```

It is less sensitive than Gini or entropy and is often used more for pruning than for split selection.

47

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Handling Continuous and Categorical Splits

For continuous attributes, sort attribute values and evaluate thresholds such as $A \leq v$ and $A > v$. Candidate thresholds are often placed between adjacent sorted values, especially where class labels change.

For categorical attributes, a split can be multiway or binary. Binary splits may require grouping categories, for example {Sports, Luxury} versus {Family}.

48

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Overfitting

Overfitting occurs when a model fits training data too closely, including noise and accidental patterns. A highly complex tree can have low training error but high test/generalization error (PDF pp. 89-96).

Training error usually decreases as tree complexity increases. Test error often decreases first, then increases after the model becomes too complex. The best model is usually not the tree with the smallest training error.

Formula - general idea (PDF pp. 97-100):

```
```text
Generalization error roughly = training error + model complexity
penalty
```
```

49

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Model Selection and MDL

The Minimum Description Length idea balances model complexity and data fit (PDF pp. 97-100). A model should compress the data well without being unnecessarily complex.

Formula - MDL cost:

```
```text
Cost(Model, Data) = Cost(Data | Model) + x * Cost(Model)
```
```

The best model has low total cost, not merely low training error.

50

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Pruning

Pruning reduces overfitting by simplifying the tree.

- Pre-pruning: stop tree growth early if a split is not useful.
- Post-pruning: grow the tree first, then remove branches that do not improve estimated generalization.

Common pruning ideas:

- Replace subtree with leaf if it improves validation performance.
- Use pessimistic error estimates.
- Use minimum records per leaf or minimum gain thresholds.

Numerical - pessimistic pruning example (PDF p. 101):

```text

Before pruning:  $(10 + 0.5) / 30 = 10.5 / 30$

After pruning:  $(9 + 4 * 0.5) / 30 = 11 / 30$

```

Because the after-pruning error estimate is larger in this example, pruning would not be preferred for that subtree.

51

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Validation and Cross- Validation

When data is limited, cross-validation estimates model performance more reliably than a single train-test split. In k-fold cross-validation, the data is divided into k folds; each fold is used once as validation while the others are used for training. Results are averaged.

Validation is used to tune model complexity, choose parameters, and decide pruning.

52

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Artificial Neural Network Intro

The session then introduces neural networks (PDF pp. 104-114). A neural network consists of connected units/neurons. Each neuron computes a weighted sum of inputs and passes it through an activation function.

Basic perceptron idea:

- Inputs are multiplied by weights.
- Weighted sum plus bias is computed.
- Activation/sign function gives output.

Formula - perceptron example (PDF p. 106):

```
'''text
Y = sign(0.3X1 + 0.3X2 + 0.3X3 - 0.4)
'''
```

Learning adjusts weights to reduce prediction error.

53

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Exam Pointers

- If asked to choose a split, compute weighted impurity for each candidate.
- Entropy and Gini are both impurity measures; gain ratio adjusts information gain.
- Overfitting is identified by training error falling while test error rises.
- Pruning is a model-complexity control method.
- ANN starts with weighted inputs, activation, and weight updates.

54

CS 4 - Decision Tree Impurity, Overfitting, Pruning, Validation, and ANN Intro: Formulae/Numericals Highlight

- Formula: $GINI(t) = 1 - \sum_i p_i(t)^2$ (PDF pp. 73-76).
- Formula: $GINI_split = \sum_i (n_i/n) GINI(i)$ (PDF p. 76).
- Formula: $Entropy(t) = -\sum_i p_i(t) \log_2 p_i(t)$ (PDF pp. 78-82).
- Formula: $Gain = Entropy(parent) - \text{weighted_entropy}(children)$ (PDF pp. 78-82).
- Formula: $Gain\ Ratio = Gain_split / SplitInfo$ (PDF pp. 82-83).
- Formula: $Error(t) = 1 - \max_i p_i(t)$ (PDF p. 83).
- Formula: $Cost(Model, Data) = Cost(Data|Model) + x * Cost(Model)$ (PDF pp. 97-100).
- Numerical: pessimistic pruning before/after calculation on PDF p. 101.
- Formula: perceptron output $Y = \text{sign}(\text{weighted sum} + \text{bias})$; example on PDF p. 106.

55

CS 5 - Neural Networks and Ensemble Methods

CS 5 - Neural Networks and Ensemble Methods

Session divider

56

CS 5 - Neural Networks and Ensemble Methods: Neural Network Building Blocks

An artificial neural network is built from processing units connected by weighted links. Each unit computes:

1. Weighted sum of inputs.
2. Adds bias/intercept.
3. Applies activation function.
4. Sends output to the next layer.

The network learns by changing weights so that predicted outputs become closer to actual labels.

57

CS 5 - Neural Networks and Ensemble Methods: Perceptron

A perceptron is a single-layer linear classifier. It can learn linearly separable patterns. Its output is often produced by a sign or threshold function.

Formula - perceptron decision (PDF p. 106):

```
```text
Y = sign(w0 + w1X1 + w2X2 + ... + wdXd)
```
```

The PDF example is:

```
```text
Y = sign(0.3X1 + 0.3X2 + 0.3X3 - 0.4)
```
```

If the weighted sum is positive, predict one class; otherwise predict the other.

58

CS 5 - Neural Networks and Ensemble Methods: Weight Learning

Weights are updated based on prediction error. The high-level idea:

```
'''text
new weight = old weight + learning_rate * correction
'''
```

The learning rate controls step size. Too small means slow learning; too large can overshoot.

59

CS 5 - Neural Networks and Ensemble Methods: Multilayer Neural Networks

Single perceptrons cannot solve non-linearly separable problems such as XOR. Multilayer networks add hidden layers. Hidden units transform the input space so that complex decision boundaries can be learned (PDF pp. 108-114).

Important terms:

- Input layer: receives attributes.
- Hidden layer: learns internal representations.
- Output layer: produces prediction.
- Activation function: introduces non-linearity.
- Loss/error function: measures prediction mistake.

60

CS 5 - Neural Networks and Ensemble Methods: Backpropagation

Backpropagation trains multilayer networks using gradient descent. It computes how much each weight contributed to the final error and updates weights in the direction that reduces loss.

Main steps:

1. Forward pass: compute outputs.
2. Error computation: compare predicted and actual output.
3. Backward pass: propagate error gradients from output layer to earlier layers.
4. Weight update: adjust weights using gradients and learning rate.

Formula - gradient descent idea:

```
```text
w <- w - learning_rate * (partial derivative of loss with respect
to w)
```
```

61

CS 5 - Neural Networks and Ensemble Methods: Ensemble Methods

An ensemble combines multiple classifiers to produce a stronger classifier. The main idea is that multiple reasonably accurate and diverse models can outperform a single model (PDF pp. 115-116).

Necessary conditions for useful ensembles:

- Base classifiers should be better than random guessing.
- Their errors should not be perfectly correlated.
- Diversity among models is important.

62

CS 5 - Neural Networks and Ensemble Methods: Bagging

Bagging, or bootstrap aggregating, trains multiple models on bootstrap samples of the training data. A bootstrap sample is created by sampling with replacement. Each model sees a slightly different data set. Final prediction is usually majority vote for classification or averaging for regression (PDF pp. 117-121).

Bagging reduces variance and is especially useful for unstable classifiers such as decision trees.

63

CS 5 - Neural Networks and Ensemble Methods: Boosting

Boosting trains models sequentially. Each new model focuses more on records that previous models misclassified. Final prediction is a weighted vote of the base classifiers (PDF pp. 122-127).

Boosting can reduce bias and variance, but it can be sensitive to noise and outliers because it emphasizes difficult records.

64

CS 5 - Neural Networks and Ensemble Methods: AdaBoost

AdaBoost assigns weights to training records. Initially all records have equal weight. After each round:

- Misclassified records get higher weights.
- Correctly classified records get lower weights.
- A classifier's vote depends on its error rate.

If an intermediate classifier has error higher than 50% in binary classification, it is worse than random guessing and should not be used directly; the PDF notes weights may be reset and resampling repeated (PDF p. 125).

Numerical: PDF p. 127 shows an AdaBoost table with changing weights across rounds and final class determined by the sign of weighted votes.

65

CS 5 - Neural Networks and Ensemble Methods: Random Forest

Random forest builds an ensemble of decision trees using two kinds of randomness (PDF p. 127):

- Bootstrap samples of records, like bagging.
- Random subset of attributes considered at each split.

Trees are usually grown deep and unpruned. The ensemble prediction is majority vote. Random forests reduce correlation among trees and usually perform strongly with little tuning.

66

CS 5 - Neural Networks and Ensemble Methods: Gradient Boosting

Gradient boosting builds models sequentially to correct residual errors of previous models. Unlike AdaBoost's record-weight view, gradient boosting is often described as functional gradient descent over a loss function. Each new model fits the negative gradient/residual of the loss.

67

CS 5 - Neural Networks and Ensemble Methods: Class Imbalance Intro

The session begins the transition to imbalanced classification (PDF pp. 129-142). In imbalanced data, one class has many more records than another. Examples include fraud detection, intrusion detection, defective products, and rare diseases. Accuracy can be misleading because a classifier can get high accuracy by predicting only the majority class.

68

CS 5 - Neural Networks and Ensemble Methods: Exam Pointers

- Perceptron is linear; multilayer networks handle nonlinear patterns.
- Backpropagation is gradient-based weight learning.
- Bagging trains independent models on bootstrap samples.
- Boosting trains sequential models and emphasizes hard examples.
- Random forest = bagging + random feature selection at splits.
- Accuracy is weak for imbalanced data.

69

CS 5 - Neural Networks and Ensemble Methods: Formulae/Numericals Highlight

- Formula: perceptron $Y = \text{sign}(w_0 + \sum_i w_i X_i)$; PDF example $Y = \text{sign}(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$ (PDF p. 106).
- Formula: gradient update $w \leftarrow w - \eta \cdot d\text{Loss}/dw$ where η is learning rate (PDF pp. 106-113).
- Numerical: AdaBoost weight/classification table on PDF p. 127.
- Conceptual numerical: binary ensemble works well when individual error rates are below 0.5 and errors are not highly correlated (PDF p. 116).

70

**CS 6 - Imbalanced Classification, KNN, Rule
Classifiers, and Association Rule Mining Intro**

**CS 6 - Imbalanced Classification, KNN,
Rule Classifiers, and Association Rule
Mining Intro**

Session divider

71

**CS 6 - Imbalanced Classification, KNN, Rule
Classifiers, and Association Rule Mining Intro: Imbalanced Class Problem**

Class imbalance occurs when one class is much rarer than another.
The minority class is often the class of interest: fraud,
intrusion, rare disease, defective item, or positive COVID test
(PDF p. 129).

Accuracy is misleading in imbalanced settings. If only 1% of
transactions are fraudulent, a model that predicts "not fraud" for
every transaction gets 99% accuracy but is useless for fraud
detection.

72

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Confusion Matrix

For binary classification:

- TP: true positives, positive records correctly predicted positive.
- TN: true negatives, negative records correctly predicted negative.
- FP: false positives, negative records incorrectly predicted positive.
- FN: false negatives, positive records incorrectly predicted negative.

The confusion matrix is the basis for precision, recall, F-measure, ROC, and AUC (PDF pp. 130-140).

73

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Classification Metrics (1/2)

Formula - Accuracy (PDF p. 130):

```
```text
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```
```

Formula - Precision:

```
```text
Precision = TP / (TP + FP)
```
```

Precision answers: among predicted positives, how many are actually positive?

Formula - Recall / True Positive Rate:

```
```text
Recall = TPR = TP / (TP + FN)
```
```

Recall answers: among actual positives, how many did the model catch?

Formula - F-measure / F1:

74

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Classification Metrics (2/2)

F1 = 2 precision recall / (precision + recall)

...

F1 balances precision and recall.

Formula - False Positive Rate:

...text

$$FPR = FP / (FP + TN)$$

...

75

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: ROC and AUC

The ROC curve plots true positive rate against false positive rate at different classification thresholds (PDF pp. 136-140). It shows the tradeoff between catching positives and producing false alarms.

AUC summarizes ROC performance:

- AUC = 1.0: ideal classifier.
- AUC = 0.5: random guessing.
- Higher AUC means better ranking ability.

76

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Handling Class Imbalance

Common strategies:

- Change threshold instead of using default 0.5.
- Oversample minority class.
- Undersample majority class.
- Use cost-sensitive learning.
- Use metrics such as precision, recall, F1, ROC/AUC instead of accuracy.
- Use stratified sampling and cross-validation.

77

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: K-Nearest Neighbor Classifier

KNN is an instance-based classifier (PDF pp. 143-149). It does not build an explicit model during training. To classify a new record:

1. Compute distance from the new record to training records.
2. Select the k nearest neighbors.
3. Assign the majority class among those neighbors.

Key choices:

- Value of k .
- Distance measure.
- Attribute scaling.
- Tie-breaking.
- Weighted or unweighted vote.

KNN is simple and flexible, but prediction can be expensive for large training sets and performance depends heavily on distance quality.

78

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Rule- Based Classifiers

Rule-based classifiers use IF-THEN rules (PDF pp. 150-163):

```
```text
IF condition THEN class
```
```

Example:

```
```text
IF Refund = No AND MaritalStatus = Single AND Income > 80K THEN
 Cheat = Yes
```
```

Rules can be extracted from decision trees or learned directly.
Rule quality depends on coverage and accuracy.

- Coverage: how many records satisfy the rule condition.
- Accuracy/confidence: among covered records, how many belong to the rule's class.

Rule ordering matters if multiple rules match. Ordered rule lists
test rules sequentially; unordered rule sets need conflict
resolution.

79

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Association Analysis Introduction

Association analysis discovers interesting relationships among
items in transaction data (PDF pp. 164-184). A transaction
contains a set of items. An itemset is a set of one or more items.

Example:

```
```text
{Diaper, Milk} -> {Beer}
```
```

This means transactions containing diaper and milk tend also to
contain beer.

80

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Support and Confidence

Support measures how frequently an itemset appears. Confidence measures how often the rule consequent appears when the antecedent appears.

Formula - support count:

```
```text
sigma(X) = number of transactions containing itemset X
```
```

Formula - support fraction:

```
```text
support(X) = sigma(X) / N
```
```

Formula - confidence (PDF pp. 165-167):

```
```text
confidence(X -> Y) = support(X union Y) / support(X)
```
```

A rule is usually considered strong if it satisfies minimum support and minimum confidence thresholds.

81

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Frequent Itemset Mining

The association rule problem is commonly split into two subproblems:

1. Find all frequent itemsets whose support is at least minsup.
2. Generate high-confidence rules from those frequent itemsets.

The first step is usually more expensive.

82

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Apriori Principle

The Apriori principle is the central pruning idea (PDF pp. 168-184):

> If an itemset is infrequent, then all of its supersets must also be infrequent.

Equivalently:

> If an itemset is frequent, then all of its subsets must be frequent.

This allows pruning of candidate itemsets before counting them.

83

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Apriori Algorithm

High-level Apriori process:

1. Find frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent 1-itemsets.
3. Count candidate support.
4. Keep frequent candidates.
5. Repeat for larger itemsets until no candidates remain.

Candidate generation uses self-join and pruning. Pruning removes candidates whose subsets are not frequent.

84

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Exam Pointers

- For imbalanced classification, never rely only on accuracy.
- In KNN, distance scaling and choice of k are critical.
- Rule classifiers are interpretable but need conflict/default handling.
- Association rules require both support and confidence.
- Apriori uses anti-monotonicity of support.

85

CS 6 - Imbalanced Classification, KNN, Rule Classifiers, and Association Rule Mining Intro: Formulae/Numericals Highlight

- Formula: $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$ (PDF p. 130).
- Formula: $\text{Precision} = TP / (TP + FP)$ (PDF pp. 130-140).
- Formula: $\text{Recall} = TP / (TP + FN)$ (PDF pp. 130-140).
- Formula: $F1 = 2pr / (p + r)$ (PDF pp. 130-140).
- Formula: $FPR = FP / (FP + TN)$ (PDF pp. 136-140).
- Numerical: AUC ideal classifier 1.0; random classifier 0.5 (PDF pp. 136-140).
- Formula: $\text{support}(X) = \sigma(X) / N$; $\text{confidence}(X \rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$ (PDF pp. 165-167).
- Conceptual formula: Apriori pruning: infrequent itemset $\rightarrow$ all supersets infrequent (PDF pp. 168-184).

86

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns

Session divider

87

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Frequent Itemset Variants

After basic Apriori, the course moves to compact representations and advanced issues.

Frequent itemset: itemset whose support is at least minsup.

Closed frequent itemset: a frequent itemset for which no proper superset has the same support. Closed itemsets preserve exact support information while reducing redundancy.

Maximal frequent itemset: a frequent itemset for which no proper superset is frequent. Maximal itemsets are more compact but do not preserve support of all subsets.

Exam distinction:

- Closed itemsets are lossless for support reconstruction.
- Maximal itemsets are more compact but lossy.

88

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Association Rule Generation

Once a frequent itemset L is found, rules are generated by splitting it into antecedent and consequent:

```
```text
A -> L - A
```
```

Only rules satisfying minimum confidence are retained.

89

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Limits of Confidence

Confidence alone can be misleading because it ignores the base rate of the consequent. If item Y is already very common, $X \rightarrow Y$ may have high confidence even when X does not actually increase the chance of Y.

Numerical - Tea/Coffee example (PDF pp. 200-203):

If:

```
```text
confidence(Tea -> Coffee) = 0.75
P(Coffee) = 0.80
```
```

Then customers who buy tea are actually less likely than average to buy coffee. The rule has high confidence but indicates negative association.

90

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Lift / Interest

Lift compares observed co-occurrence with expected co-occurrence under independence.

Formula - Lift / Interest (PDF pp. 200-203):

```
```text
Lift(X -> Y) = P(X,Y) / (P(X)P(Y))
= P(Y|X) / P(Y)
```
```

Interpretation:

- Lift > 1: positive association.
- Lift = 1: independence.
- Lift < 1: negative association.

Numerical - Tea/Coffee interest:

```
```text
Lift = 0.75 / 0.80 = 0.9375
```
```

Since lift is below 1, tea and coffee are negatively associated in this example.

91

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Correlation and Statistical Interestingness

The lecture discusses that interestingness can be objective or subjective. Objective measures include support, confidence, lift, correlation, conviction, and statistical tests. Subjective interestingness depends on user expectations, actionability, novelty, and domain usefulness.

Correlation-style measures help distinguish real association from high-confidence rules caused by common consequents.

92

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Simpson's Paradox

Simpson's paradox occurs when an association observed in aggregated data reverses or disappears when data is split into subgroups (PDF pp. 204-207). This matters because association rules can be misleading if confounding variables exist.

Exam point: always consider whether a discovered rule is stable across relevant subgroups.

93

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Support Skew and Rare Items

Support-based pruning can miss rare but important patterns. For example, expensive items or rare medical events may have low support but high value. If minsup is too high, rare-item rules disappear. If minsup is too low, too many uninteresting patterns appear.

This is the support-skew problem.

94

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Cross- Support Pattern

Cross-support patterns contain items with very different support levels. Such patterns are often spurious because a very frequent item can co-occur with many rare items by chance.

Formula - support ratio (PDF p. 212):

```
```text
r(X) = min_i s(x_i) / max_i s(x_i)
```
```

If $r(X)$ is very small, the itemset has strong support imbalance.

95

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: h-Confidence and Hyperclique Patterns

h-confidence is used to detect strong affinity patterns while avoiding cross-support problems.

Formula - h-confidence (PDF pp. 213-214):

```
```text
hconf(X) = s(X) / max_i s(x_i)
```
```

Property:

```
```text
hconf(X) <= r(X)
```
```

Hyperclique patterns require high h-confidence and are useful for finding strongly associated items even with support skew.

96

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Association Patterns With Different Data Types

Association mining can be extended beyond simple binary baskets:

- Categorical attributes: map attribute-value pairs to items.
- Continuous attributes: discretize into intervals before mining.
- Multi-level associations: mine at different concept hierarchy levels, such as milk -> dairy -> food.
- Multi-dimensional associations: combine multiple attributes such as age group, income range, and product.

97

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Sequential Pattern Mining

Sequential pattern mining finds frequent ordered patterns. Unlike itemsets, order matters.

Example:

```
``text
<buy laptop> -> <buy laptop bag> -> <buy warranty>
``
```

Applications include customer purchase sequences, web clickstreams, biological sequences, telecom alarms, and event logs.

Key difference:

- Association rule: co-occurrence in a transaction.
- Sequential pattern: ordered occurrence across time/events.

98

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Subgraph Mining

Graph pattern mining finds recurring substructures in graph data (PDF pp. 230-260). Graph data contains vertices and edges, sometimes with labels and weights.

Applications:

- Chemical compound mining.
- Protein interaction networks.
- Social networks.
- Web graphs.
- Communication networks.

Important challenge: graph isomorphism. Two graphs may look different due to vertex naming but have the same structure. Mining frequent subgraphs requires checking structural equivalence, which can be computationally expensive.

99

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Clustering Transition

The session begins moving toward clustering by connecting pattern discovery with grouping and structure discovery. Frequent patterns describe co-occurrence; clustering groups similar objects. Both are descriptive tasks, but clustering does not require predefined labels or explicit itemset rules.

100

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Exam Pointers

- Confidence is not enough; compare against consequent base rate.
- Lift below 1 means negative association even if confidence is high.
- Closed versus maximal itemsets is a common exam distinction.
- Support skew explains why a single minsup can be problematic.
- Sequential patterns require order; association rules do not.
- Subgraph mining is harder due to graph isomorphism.

101

CS 7 - Frequent Pattern Mining, Interestingness, Support Skew, Sequential and Graph Patterns: Formulae/Numericals Highlight

- Formula: $\text{confidence}(X \rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$ (review from CS 6).
- Formula: $\text{Lift} = P(X, Y) / (P(X)P(Y)) = P(Y|X) / P(Y)$ (PDF pp. 200-203).
- Numerical: Tea $\rightarrow$ Coffee confidence 0.75, baseline $P(\text{Coffee}) = 0.8$, lift 0.9375, so negative association (PDF pp. 200-203).
- Formula: $r(X) = \min_i s(x_i) / \max_i s(x_i)$ for cross-support ratio (PDF p. 212).
- Formula: $\text{hconf}(X) = s(X) / \max_i s(x_i)$ and $\text{hconf}(X) \leq r(X)$ (PDF pp. 213-214).
- Formula/Numerical: Statistical association test appears in PDF pp. 224-225:

```
```text
Z = (mu - mu' - Delta) / sqrt(s1^2/n1 + s2^2/n2)
```
```

The PDF example reports $Z = 3.11 > 1.64$, so the tested association is statistically significant at the stated one-sided threshold.

102

CS 8 - Cluster Analysis

CS 8 - Cluster Analysis

Session divider

103

CS 8 - Cluster Analysis: What Clustering Does

Clustering groups data objects so that:

- Intra-cluster similarity is high.
- Inter-cluster similarity is low.

Unlike classification, clustering is unsupervised: there are no predefined class labels. The algorithm discovers group structure from the data (PDF pp. 261-264).

Applications:

- Customer segmentation.
- Document grouping.
- Gene/protein grouping.
- Image segmentation.
- Climate pattern discovery.
- Outlier detection support.
- Data summarization.

104

CS 8 - Cluster Analysis: Types of Clustering

Clustering methods can be categorized by the type of cluster they seek (PDF pp. 261-269):

- Partitional: divide data into non-overlapping clusters, such as k-means.
- Hierarchical: build nested clusters, either agglomerative or divisive.
- Density-based: clusters are dense regions separated by sparse regions, such as DBSCAN.
- Graph-based: clusters are strongly connected components or graph partitions.
- Model-based: assume data is generated by a mixture of probability distributions.
- Subspace: find clusters in subsets of dimensions.
- Fuzzy: allow partial membership in multiple clusters.

105

CS 8 - Cluster Analysis: K-Means Clustering

K-means is a partitional clustering algorithm. It assumes clusters are represented by centroids and works best for globular, roughly equal-sized clusters in numeric Euclidean space (PDF pp. 270-280).

Algorithm:

1. Choose k initial centroids.
2. Assign each point to the nearest centroid.
3. Recompute each centroid as the mean of assigned points.
4. Repeat assignment and update until convergence.

K-means minimizes within-cluster sum of squared errors.

Formula - SSE objective (PDF p. 271):

```
'''text
SSE = sum_i sum_{x in C_i} dist(x, m_i)^2
'''
```

where m_i is the centroid of cluster C_i .

106

CS 8 - Cluster Analysis: K-Means Strengths and Weaknesses

Strengths:

- Simple and efficient.
- Works well for compact, spherical clusters.
- Scales to large numeric data.

Weaknesses:

- Must choose k.
- Sensitive to initialization.
- Sensitive to outliers.
- Not good for arbitrary-shaped clusters.
- Struggles with clusters of different size/density.
- Requires meaningful mean and distance.

107

CS 8 - Cluster Analysis: Initialization and Empty Clusters

Different initial centroids can lead to different final clusters. Bad initialization can produce poor local minima. Methods like multiple restarts or k-means++ improve initialization. Empty clusters can occur when no points are assigned to a centroid; common fixes include removing that centroid or reinitializing it to a high-error point.

108

CS 8 - Cluster Analysis: Bisecting K-Means

Bisecting k-means is a divisive method:

1. Start with all points in one cluster.
2. Repeatedly choose one cluster to split into two using k-means with $k=2$.
3. Continue until desired number of clusters is reached.

It often produces better results than ordinary k-means and can be seen as combining partitional and hierarchical ideas.

109

CS 8 - Cluster Analysis: Hierarchical Clustering

Hierarchical clustering creates a tree of clusters called a dendrogram (PDF pp. 281-295).

Two main types:

- Agglomerative: start with each point as its own cluster, then repeatedly merge closest clusters.
- Divisive: start with all points in one cluster, then repeatedly split.

Agglomerative hierarchical clustering depends heavily on the inter-cluster distance definition:

- MIN / single link: distance between closest pair of points.
- MAX / complete link: distance between farthest pair.
- Group average: average pairwise distance.
- Centroid: distance between centroids.
- Ward's method: merge that causes minimum increase in SSE.

Single link can handle non-elliptical shapes but can suffer chaining. Complete link produces compact clusters but may break large clusters. Average link is a compromise.

110

CS 8 - Cluster Analysis: DBSCAN

DBSCAN is density-based clustering (PDF pp. 296-299). It groups dense regions and marks sparse-region points as noise.

Parameters:

- Eps: neighborhood radius.
- MinPts: minimum number of points required to form a dense region.

Point types:

- Core point: has at least MinPts points within Eps.
- Border point: not core, but lies within Eps of a core point.
- Noise point: neither core nor border.

DBSCAN can find arbitrary-shaped clusters and detect noise, but it struggles when clusters have varying densities and parameters are hard to choose.

111

CS 8 - Cluster Analysis: Cluster Validity

Cluster validity evaluates whether clustering results are meaningful (PDF pp. 300-309).

Types:

- Internal validation: uses only data and clustering result, e.g., SSE, silhouette.
- External validation: compares with known labels, if available.
- Relative validation: compares multiple clusterings or parameter choices.

112

CS 8 - Cluster Analysis: SSE, SSB, and Cluster Quality

SSE measures within-cluster compactness. SSB measures between-cluster separation. In the PDF example, total variation is constant and split between SSE and SSB (PDF p. 302).

Numerical - PDF example:

```
```text
k = 1: SSE = 10
k = 2: SSE = 1, SSB = 9
Total = SSE + SSB = 10
```
```

As k increases, SSE usually decreases. The challenge is to choose k where improvement is meaningful, not just because more clusters always reduce SSE.

113

CS 8 - Cluster Analysis: Silhouette Coefficient

Silhouette measures how well an object fits its assigned cluster compared with the nearest other cluster (PDF p. 303).

Formula - silhouette:

```
```text
s = (b - a) / max(a, b)
```
```

where:

- a : average distance from the point to other points in same cluster.
- b : minimum average distance from the point to points in another cluster.

Interpretation:

- close to 1: well-clustered.
- around 0: near boundary.
- negative: possibly assigned to wrong cluster.

114

CS 8 - Cluster Analysis: Advanced Clustering Topics

The PDF also includes advanced clustering methods (PDF pp. 885-929):

- Fuzzy c-means: each point has membership weights in clusters instead of hard assignment.
- EM clustering: model-based clustering using mixture distributions, often Gaussian mixtures.
- Self-organizing maps (SOM): neural-network based topology-preserving clustering/visualization.
- CLIQUE: subspace clustering for high-dimensional data.
- DENCLUE: density-based clustering using density estimation.
- Chameleon: graph-based hierarchical clustering using relative interconnectivity and relative closeness.
- Spectral clustering: graph partitioning using eigenvectors of the graph Laplacian.
- SNN clustering: shared nearest neighbor graph plus density-based ideas.

115

CS 8 - Cluster Analysis: Spectral Clustering

Spectral clustering converts data into a graph and partitions it using eigenvectors (PDF pp. 911-914).

Formula - graph Laplacian:

```
```text
L = D - W
```
```

where W is the weighted adjacency/proximity matrix and D is the diagonal degree matrix. The algorithm uses the first k eigenvectors of L and applies k -means to rows of that eigenvector matrix.

116

CS 8 - Cluster Analysis: Exam Pointers

- K-means minimizes SSE and assumes centroid makes sense.
- Hierarchical clustering depends on linkage choice.
- DBSCAN uses density and can mark noise.
- Silhouette combines cohesion and separation.
- More clusters reduce SSE, so use validity methods to select k.
- Spectral clustering is graph-based and uses Laplacian eigenvectors.

117

CS 8 - Cluster Analysis: Formulae/Numericals Highlight

- Formula: $SSE = \sum_i \sum_{\{x \in C_i\}} \text{dist}(x, m_i)^2$ (PDF p. 271).
- Numerical: k=1: SSE=10; k=2: SSE=1, SSB=9, total 10 (PDF p. 302).
- Formula: $s = (b-a)/\max(a,b)$ for silhouette, range [-1,1] (PDF p. 303).
- Formula: DBSCAN uses Eps and MinPts; core point has at least MinPts neighbors within Eps (PDF pp. 296-299).
- Formula: spectral clustering Laplacian $L = D - W$ (PDF pp. 912-914).

118

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing

Session divider

119

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: What Is an Anomaly?

An anomaly is a data object that deviates significantly from normal or expected behavior (PDF pp. 310-315). Other terms include outlier, novelty, exception, deviation, or abnormal point.

Applications:

- Credit-card fraud.
- Network intrusion.
- Medical diagnosis.
- Industrial fault detection.
- Sensor monitoring.
- Cybersecurity.
- Event/change detection.

120

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Types of Anomalies

Common anomaly types:

- Point anomaly: a single object is abnormal.
- Contextual anomaly: object is abnormal only in a specific context, such as high temperature in winter.
- Collective anomaly: a group/sequence is abnormal even if individual points are not.

Context matters. The same value can be normal in one setting and anomalous in another.

121

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Challenges in Anomaly Detection

Anomaly detection is hard because:

- Anomalies are rare.
- Normal behavior can change over time.
- Labels are often unavailable.
- Noise can look like anomalies.
- Attackers may adapt to detection.
- High-dimensional data weakens distance/density intuition.

122

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Statistical Anomaly Detection

Statistical methods assume a data distribution and flag objects unlikely under that distribution. For example, if data is Gaussian, points far from the mean may be anomalous.

Formula - Grubbs statistic (PDF p. 320):

```
```text
G = max |X - Xbar| / s
```
```

where Xbar is sample mean and s is sample standard deviation. If G exceeds the critical threshold, reject the null hypothesis that there is no outlier.

123

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Distance-Based Anomaly Detection

Distance-based methods say a point is anomalous if it is far from most other points. A point may be flagged if fewer than a specified number of points lie within distance d, or if its k-nearest-neighbor distance is very large.

Strength: simple and intuitive.

Weakness: distance becomes less meaningful in high dimensions and global distance thresholds fail with varying densities.

124

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Density-Based Anomaly Detection

Density-based methods compare local density around a point with densities around neighboring points (PDF pp. 331-333). A point in a low-density region relative to neighbors is suspicious.

Formula - simple density idea (PDF p. 331):

```
```text
density(x) = 1 / dist(x, k)
```
```

where $\text{dist}(x, k)$ is distance from x to its k th nearest neighbor.

Relative density compares a point's density with neighborhood density. Local Outlier Factor (LOF) formalizes this idea (PDF p. 333).

Interpretation:

- LOF around 1: similar density to neighbors.
- LOF much greater than 1: potential outlier.

125

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Clustering-Based Anomaly Detection

Clustering-based methods treat small clusters, distant points, or points far from centroids as anomalies. For k-means, high distance from assigned centroid can indicate anomaly. For DBSCAN, noise points are natural anomaly candidates.

Be careful: a small cluster may be a valid rare group, not an anomaly.

126

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Reconstruction-Based Anomaly Detection

Reconstruction methods learn a compressed representation of normal data. Objects with high reconstruction error are anomalies (PDF p. 340). PCA and autoencoders are common examples.

Formula - reconstruction error:

```
```text
error = ||x - x_hat||
```
```

where x_{hat} is reconstructed version of x .

127

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Web/Data Mining Trends

The lecture also transitions toward modern data mining applications: web data, social media, recommendations, streams, text, graphs, and cyber applications. The key idea is that data mining models must operate in real environments where data is large, dynamic, heterogeneous, networked, and often user-driven.

The PDF's "Data Mining Trends and Research Frontiers" block lists complex data types that extend beyond simple tables: sequence data, time series, symbolic sequences, biological sequences, graphs/networks, spatial and spatiotemporal data, cyber-physical system data, multimedia, text, web data, and streams (PDF pp. 358-362).

128

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Sequence, Time-Series, and Biological Mining Trends

Sequence and time-series mining includes similarity search, subsequence matching, dimensionality reduction, query-based similarity search, motif-based similarity search, regression, trend analysis, sequential pattern mining, sequence classification, and biological sequence alignment (PDF p. 360).

For time series, the PDF separates long-term trend, cyclic variation, seasonal variation, and random movement. For biological sequences, the PDF mentions pairwise and multiple-sequence alignment, substitution matrices, BLAST, Markov chains, Hidden Markov Models, and forward/Viterbi/Baum-Welch algorithms (PDF p. 360).

129

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Graph and Network Mining Trends

The PDF explicitly lists graph pattern mining, frequent subgraph patterns, closed graph patterns, gSpan, CloseGraph, statistical modeling of networks, small-world behavior, power-law/log-tail distributions, densification, graph clustering/classification, heterogeneous network ranking/classification, role discovery, link prediction, similarity search, network OLAP, and evolution of social/information networks (PDF p. 361).

This is the bridge from earlier frequent subgraph mining to later CS13-CS14 dynamic graph mining.

130

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Statistical Data Mining Methods (1/2)

The PDF also frames data mining through statistical methodology (PDF pp. 364-370). Important methods:

- Regression: predict numeric response from one or more numeric predictors.
- Generalized Linear Models: connect predictors to responses such as categorical or count outcomes; includes logistic and Poisson regression.
- Mixed-effect models: handle grouped data and repeated/grouped observations.
- Regression trees: tree models where leaves predict numeric means.
- Analysis of variance: compare numeric response across categorical factors.
- Factor analysis: discover latent variables/factors behind observed variables.
- Discriminant analysis: classify using functions that separate groups.
- Time-series models: autoregression, ARIMA, and long-memory models.
- Survival analysis: predict probability of surviving beyond time t.

Exam point: these are not separate from data mining; they are

131

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Statistical Data Mining Methods (2/2)

statistical foundations or tools used inside mining workflows.

132

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Views on Data Mining Foundations

The PDF presents multiple "foundation" views (PDF pp. 372-373):

- Data reduction view: mining reduces representation while preserving useful information.
- Compression view: patterns such as rules, trees, clusters, and models compress data.
- Probability/statistical view: mining discovers joint probability structure.
- Microeconomic/utility view: patterns matter when they support useful decisions.
- Inductive database view: patterns become queryable objects along with raw data.

These views explain why compression, MDL, model selection, interestingness, and decision usefulness repeatedly appear in the course.

133

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Visual and Audio Data Mining

Visual data mining uses visualization to discover patterns, trends, relationships, irregularities, and useful regions in large data (PDF pp. 375-386). It includes:

- Data visualization.
- Data mining result visualization, such as decision trees, rules, clusters, outliers, and scatter plots.
- Data mining process visualization.
- Interactive visual mining, where users guide model choices.

Audio data mining maps data patterns into sound so that unusual values, rhythms, or patterns can be heard rather than only seen (PDF p. 388).

134

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Applications and Society

Application areas in the PDF include finance, retail, telecom, science/engineering, intrusion detection, and recommender systems (PDF pp. 390-397).

Important examples:

- Finance: warehouses, credit scoring, loan payment prediction, money laundering detection, linkage analysis, classification, clustering, outlier analysis, sequential patterns (PDF pp. 391-392).
- Retail/telecom: buying behavior, shopping trends, customer retention, campaign analysis, recommendation, fraud analysis, visualization (PDF pp. 393-394).
- Science/engineering: preprocessing data from inconsistent sources, spatiotemporal/biological data, graph/network mining, domain knowledge, visualization (PDF p. 395).
- Intrusion detection: signature-based detection versus anomaly-based profiles; association/correlation/discriminative patterns; stream outlier detection; distributed mining; visualization/query tools (PDF p. 396).
- Recommender systems: content-based, collaborative, hybrid, user-item rating prediction, memory-based KNN, model-based probabilistic/clustering/Bayesian approaches (PDF p. 397).

135

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Ubiquitous, Invisible, Privacy- Preserving Mining

Ubiquitous data mining means data mining appears everywhere, such as online shopping and CRM. Invisible data mining means mining functions are embedded into daily systems, such as search ranking, without users directly seeing the mining process (PDF p. 399).

Privacy and security issues include unconstrained access to individual records, sensitive information, and output patterns that may reveal private information. The PDF lists privacy methods (PDF pp. 400-401):

- Remove sensitive identifiers.
- Use security controls such as multilevel access and encryption.
- Use privacy-preserving data mining.
- Randomization/perturbation: add noise to mask values.
- k-anonymity: each record should be indistinguishable from at least k-1 others.
- l-diversity: groups should contain diverse sensitive values.
- Distributed privacy preservation: mine horizontally/vertically partitioned data without exposing all raw data.
- Hide or distort mining results that reveal sensitive patterns.

136

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Predictive Analytics in Social Media and Big Data

PDF pp. 408-422 introduce predictive analytics in social media and big-data analytics. The important distinction is descriptive versus predictive analytics:

- Descriptive analytics: summarizes what happened or what structure exists, such as clusters, trends, dashboards, communities, or frequent patterns.
- Predictive analytics: estimates what will happen or what label/score should be assigned, such as click probability, churn risk, fraud probability, recommendation score, or future demand.

The summary slide also mentions Neuro-Symbolic AI (PDF p. 422), which combines neural learning with symbolic knowledge/rules. In the course context, treat this as a modern direction where learned representations are combined with explicit knowledge structures, constraints, or reasoning.

137

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Hypothesis Testing and Multiple Comparisons

When many patterns are tested, some will appear significant by chance. This is important for association rules, anomaly detection, and large-scale mining (PDF pp. 930-946).

If each test has false positive probability α , then with many tests the chance of at least one false positive can become large.

Formula - family-wise error rate (PDF p. 943):

```
```text
FWER = 1 - (1 - alpha)^m
```
```

where m is number of tests.

Numerical (PDF p. 943):

```
```text
alpha = 0.05, m = 10
FWER = 1 - 0.95^10 approx 0.401
```
```

So, ten tests at 5% significance create about 40.1% chance of at least one false positive. The PDF slide states 0.60 on p. 943, but 0.60 is approximately the probability of no false positive,

138

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Bonferroni Correction

Bonferroni controls family-wise error by using a stricter threshold:

Formula - Bonferroni threshold (PDF p. 943):

```
```text
alpha* = alpha / m
```
```

This reduces false positives but can be conservative and may increase false negatives.

139

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: False Discovery Rate and Benjamini-Hochberg

False Discovery Rate (FDR) controls expected proportion of false discoveries among reported discoveries (PDF pp. 944-945).

Formula - FDR quantity:

```
```text
Q = FP / (TP + FP)
```
```

when at least one positive discovery is reported.

Benjamini-Hochberg ranks p-values and chooses a cutoff that controls FDR less conservatively than Bonferroni.

140

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Exam Pointers

- Anomaly detection depends on the definition of normal.
- Distance-based and density-based outliers differ.
- LOF compares local density to neighbor density.
- Reconstruction error is central for PCA/autoencoder anomaly detection.
- Multiple testing creates many false positives if uncorrected.
- Bonferroni controls FWER; Benjamini-Hochberg controls FDR.

141

CS 9 - Anomaly Detection, Web/Data Mining Trends, and Statistical Testing: Formulae/Numericals Highlight

- Formula: $G = \max |X - \bar{X}|/s$ for Grubbs statistic (PDF p. 320).
- Formula: $\text{density}(x) = 1/\text{dist}(x, k)$ as k-distance density idea (PDF p. 331).
- Formula: LOF is based on local reachability density ratio; high LOF means more outlier-like (PDF p. 333).
- Formula: reconstruction error $\|x - \hat{x}\|$ (PDF p. 340).
- Formula: $\text{FWER} = 1 - (1 - \alpha)^m$ (PDF p. 943).
- Numerical: $\alpha = .05$, $m = 10$, $\text{FWER} = 1 - .95^{10}$ approx .401; PDF p. 943 prints .60, which matches $.95^{10}$ rather than FWER.
- Formula: Bonferroni $\alpha^* = \alpha/m$ (PDF p. 943).
- Formula: $Q = \text{FP}/(\text{TP} + \text{FP})$ for false discovery proportion/FDR idea (PDF pp. 944-945).

142

CS 10 - Web Intelligence, PageRank, and Recommender Systems

CS 10 - Web Intelligence, PageRank, and Recommender Systems

Session divider

143

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Web Intelligence and Collective Intelligence

The session begins with internet/web data as a source of large-scale intelligence. Web intelligence means aggregating and processing distributed internet information to infer useful knowledge. Collective intelligence refers to patterns that emerge from many users, pages, links, clicks, likes, shares, comments, ratings, and other interactions (PDF pp. 423-430).

Recommendation is framed as matching users with items. Items may be news articles, videos, products, webpages, posts, or advertisements. Users have preferences inferred from behavior, profile, context, location, history, and feedback.

144

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Web Mining Types

Web mining can be divided into:

- Web content mining: mining text, images, video, and multimedia content on webpages.
- Web structure mining: mining hyperlinks and graph structure.
- Web usage mining: mining user behavior such as clicks, sessions, browsing paths, and logs.

Recommender systems often combine all three.

Web content mining analyzes page text, multimedia, structured content, entity/concept resolution, relevance, ranking, summaries, and surface/deep web content. Web structure mining analyzes hyperlinks and page structure such as HTML/XML tag structure. Web usage mining extracts useful behavior from server logs, clickstreams, ratings, reviews, browsing patterns, and user feedback (PDF pp. 423-428).

145

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Web Crawlers, Search, and Spam

Web crawlers, also called spiders or robots, automatically download web pages so that search engines or mining systems can index and analyze them (PDF pp. 433-436). Universal crawlers aim to crawl broadly across the web; preferential crawlers focus on pages relevant to a particular application, competitor, topic, or business-intelligence need (PDF pp. 435-437).

Search uses crawled pages to answer keyword queries. Quality and relevance depend on both content and link structure (PDF pp. 429-432).

Search spam tries to manipulate ranking and reduce search quality. Search engines respond with reputation-based and link-analysis methods, while SEO attempts to improve page visibility by understanding or reverse-engineering ranking behavior (PDF pp. 438-439).

146

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Search, Ranking, and PageRank

PageRank ranks webpages using the hyperlink graph (PDF pp. 440-445). The core idea is that a page is important if important pages link to it.

Random-surfer intuition:

- A user randomly follows links from page to page.
- Occasionally the user jumps to a random page.
- The long-run probability of landing on each page is its PageRank.

Important points:

- More in-links usually increase rank.
- Links from high-quality/high-rank pages matter more.
- Link structure is treated as a graph.
- PageRank is recursive: a page's rank depends on ranks of pages linking to it.

147

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Recommendation Workflow

The lecture describes a recommendation workflow using news/web data:

1. Collect item data: title, body, category, topic, abstract, ID, metadata.
2. Collect user data: user ID, location, browsing behavior, queries, timing, device, clicks, dwell time.
3. Preprocess and represent both users and items.
4. Build a matching model using classification, clustering, link prediction, collaborative filtering, or other learning methods.
5. Rank items for each user.
6. Evaluate with recommendation metrics and user feedback.

This is a many-to-many matching problem: many users can match many items. A bipartite graph is a natural representation, with users on one side, items on the other, and edges representing interaction or preference.

148

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Content-Based Filtering

Content-based recommendation uses item attributes and user profiles. If a user likes certain item features, recommend similar items.

Example:

- User reads sports news.
- System builds a user profile with sports-related terms/topics.
- Recommend other sports articles with similar content.

Strengths:

- Works with item metadata/content.
- Can recommend new items if content is available.

Weaknesses:

- May overspecialize.
- Requires good content representation.
- May not capture collaborative social signals.

149

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Collaborative Filtering

Collaborative filtering uses user-item interaction patterns (PDF pp. 506-520). The central object is the utility matrix: rows are users, columns are items, and entries are ratings/clicks/preferences.

Formula - model view (PDF p. 506):

```
'''text
Model = f(utility matrix)
'''
```

Types:

- User-user collaborative filtering: find similar users and recommend what they liked.
- Item-item collaborative filtering: find similar items based on users who interacted with them.
- Model-based collaborative filtering: learn latent factors or predictive models from the utility matrix.

Similarity can be computed using cosine similarity, correlation, Euclidean distance, or learned metrics.

150

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Hybrid Recommendation

Hybrid systems combine content-based and collaborative filtering. This helps handle limitations:

- Cold-start users: little interaction history.
- Cold-start items: new item has no interactions.
- Sparse utility matrix.
- Overspecialization in content-only systems.

Hybrid systems may include user metadata, item metadata, graph structure, text representations, and behavior signals.

151

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Deep Representation and Graph-Based Recommendation

The transcript emphasizes transforming raw data into a representation/search space. For web/news/video recommendation, raw text and user behavior are converted into feature vectors or embeddings. Deep learning can learn these representations automatically.

Graph-based recommendation treats users and items as nodes in a bipartite graph. Link prediction becomes the recommendation task: predict which user-item edges should exist.

Graph neural networks can learn from graph structure across layers and produce recommendations based on learned representations of users, items, and neighborhoods.

152

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Feedback Signals

User feedback can be explicit or implicit:

- Explicit: rating, like/dislike, subscribe.
- Implicit: click, dwell time, watch time, scroll behavior, purchase, repeat visit.

Implicit feedback is noisy but abundant. Recommendation models must interpret it carefully.

153

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Recommender Evaluation Metrics

The PDF includes recommendation metrics for news and web recommendation (PDF pp. 486-489):

- Precision: among recommended items, how many were relevant/clicked.
- Recall: among relevant items, how many were recommended.
- F1: harmonic mean of precision and recall.
- AUC: ranking quality.
- Mean Average Precision (MAP): ranking precision across relevant results.
- Diversity: recommendations are not all too similar.
- Novelty: recommendations are not only obvious/popular items.
- Fairness: exposure is balanced across users/items/providers.
- Popularity bias: tendency to recommend already popular items too much.

154

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Computational Advertising

Recommendation concepts also apply to ads. The goal is to match ads to users/context while optimizing click probability, conversion, revenue, relevance, and user experience. This uses user profiling, item/ad representation, ranking, and feedback loops.

155

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Associative and Frequent-Pattern Recommenders

Frequent itemsets and association rules can recommend items that co-occur with a user's current items or history (PDF pp. 540-550).

Formula - rule score (PDF p. 546):

```
'''text
score(rule k) = support(rule k) * confidence(rule k)
'''
```

If a user's current basket/profile matches a rule antecedent, the consequent can be recommended, with rules ranked by score.

156

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Exam Pointers

- Web mining includes content, structure, and usage.
- PageRank uses graph links and recursive importance.
- Collaborative filtering depends on user-item interactions.
- Content-based filtering depends on item features and user profile.
- Hybrid methods combine signals.
- Recommendation is often a ranking problem, not just classification.

157

CS 10 - Web Intelligence, PageRank, and Recommender Systems: Formulae/Numericals Highlight

- Conceptual formula: PageRank = page importance from in-links weighted by linking-page quality (PDF pp. 440-445).
- Formula: $\text{Model} = f(\text{utility matrix})$ for collaborative filtering (PDF p. 506).
- Formula: recommendation $F1 \ 2pr/(p+r)$ when precision/recall are used (PDF pp. 486-489).
- Formula: frequent-pattern recommender $\text{score}(\text{rule } k) = \text{support}(\text{rule } k) * \text{confidence}(\text{rule } k)$ (PDF p. 546).
- Graph formulation: recommendation as link prediction on user-item bipartite graph (PDF pp. 506-561).

158

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News

Session divider

159

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Recommendation Beyond Basic Collaborative Filtering

The lecture expands recommendation into broader real-world settings:

- Cross-domain recommendation.
- Content-based filtering.
- Hybrid recommendation.
- Graph-based recommendation.
- Text-aware recommendation.
- Fake-news detection and credibility modeling.
- Descriptive and predictive recommendation use cases.

Cross-domain recommendation uses knowledge from one domain to improve recommendations in another. For example, viewing behavior on videos may help recommend articles, or product purchase patterns may help recommend services.

160

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Content-Based Recommendation Revisited

Content-based recommendation depends heavily on text and metadata representation. For documents/news:

- Tokenize text.
- Remove stop words.
- Normalize/stem/lemmatize terms if needed.
- Create term vectors or embeddings.
- Build user profile from consumed items.
- Rank new items by similarity to user profile.

This connects CS 11 to text mining.

161

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Text Mining Pipeline

Text data is unstructured. It must be transformed before mining.

Typical steps:

1. Document collection.
2. Tokenization.
3. Stop-word removal.
4. Stemming/lemmatization.
5. Term weighting.
6. Vector-space representation.
7. Modeling: classification, clustering, topic modeling, retrieval, recommendation.

Text mining tasks include document classification, document clustering, topic discovery, sentiment analysis, fake-news detection, information retrieval, and recommendation.

162

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Vector Space Model and Term Weighting

Documents can be represented as vectors where dimensions are terms. Values may be binary presence/absence, term frequency, TF-IDF, or learned embeddings.

TF-IDF intuition:

- A term is important if it appears often in a document.
- A term is less useful if it appears in almost every document.

This helps distinguish topic-specific words from common words.

163

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Similarity in Text

Cosine similarity is commonly used for text vectors because document length varies.

Formula - cosine similarity:

```
'''text
cosine(x, y) = (x dot y) / (||x|| ||y||)
'''
```

Use cosine when direction/profile matters more than raw magnitude.

164

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Latent Semantic Analysis

Latent Semantic Analysis (LSA) reduces document-term matrix dimensionality using matrix factorization, commonly Singular Value Decomposition. It captures latent concepts by mapping terms and documents into a lower-dimensional semantic space.

Benefits:

- Reduces noise.
- Captures synonymy/related terms better than raw term matching.
- Helps retrieval, clustering, and recommendation.

Limitation:

- Topics may be hard to interpret directly.

165

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Topic Modeling and LDA

Latent Dirichlet Allocation (LDA) is a probabilistic topic model (PDF pp. 562-619). It represents:

- Each document as a mixture of topics.
- Each topic as a distribution over words.

Example:

- Topic 1: cricket, match, score, team.
- Topic 2: market, stock, price, index.
- A document can be 70% sports and 30% finance.

LDA is useful for discovering hidden themes in large document collections and for representing items/users in recommenders.

The PDF notes LDA as an unsupervised topic model with soft clustering behavior, dimensionality reduction, probabilistic word-topic structure, and mixture-model/EM-style interpretation (PDF pp. 608-612). Inference/learning methods mentioned include Gibbs sampling, Expectation-Maximization, and variational Bayes (PDF p. 612).

166

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Descriptive and Predictive Recommendations

Recommendation can be descriptive or predictive:

- Descriptive: explain user/item groups, topic interests, patterns, and co-occurrences.
- Predictive: predict which item a user will click, rate, buy, watch, or read.

The same system may use descriptive models for interpretation and predictive models for ranking.

167

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Fake News and Credibility Mining (1/2)

Fake-news detection can use:

- Content features: language, claims, topics, sentiment.
- Source features: publisher credibility, author history.
- User features: propagation behavior, engagement.
- Network features: sharing graph, communities, diffusion patterns.
- Temporal features: speed and shape of spread.

This becomes a classification/anomaly/graph mining problem depending on labels and formulation.

Important caution: fake-news detection is not only text classification; context, source, propagation, and user behavior can matter.

The PDF defines knowledge in this setting as (Subject, Predicate, Object) triples. A fact is an SPO triple verified as true; a knowledge base is a set of facts; a knowledge graph represents entities as nodes and predicates/relationships as edges (PDF pp. 597-598). Knowledge bases may have redundancy, invalidity, conflicts, unreliability, and incompleteness.

Fake-news features and methods listed in the PDF include lexicon, syntax, discourse, semantics, probabilistic context-free grammars,

168

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Fake News and Credibility Mining (2/2)

generative context-sensitive grammars, frequent pattern mining, latent factor modeling, multimodal feature extractors, event discriminators in a min-max setup, and fake-news classifiers over multimodal/relational information (PDF pp. 599-600).

169

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Recommendation Bias and Feedback Loops

Recommender systems can amplify popularity, create filter bubbles, and reinforce existing user preferences. Evaluation should include diversity, novelty, fairness, and long-term behavior, not only immediate clicks.

170

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Exam Pointers

- Text mining requires representation before modeling.
- LSA is matrix-factorization based; LDA is probabilistic topic modeling.
- Content-based recommendation depends on item/user feature similarity.
- Fake-news detection can combine text, source, user, network, and temporal features.
- Recommendation can be descriptive, predictive, or both.

171

CS 11 - Recommendation Extensions, Text Mining, Topic Modeling, and Fake News: Formulae/Numericals Highlight

- Formula: cosine similarity $\text{cosine}(x,y) = (x \cdot y) / (||x|| ||y||)$ for text/content similarity.
- Conceptual formula: LDA: document = mixture of topics; topic = distribution over words (PDF pp. 562-619).
- Conceptual numerical: User/item ranking can use predicted relevance score; evaluation can use precision, recall, F1, AUC, MAP, diversity, and novelty (PDF pp. 486-489).

172

CS 12 - Data Stream Mining

CS 12 - Data Stream Mining

Session divider

173

CS 12 - Data Stream Mining: What Is a Data Stream?

A data stream is a continuous, potentially unbounded sequence of data arriving over time. Examples include web clicks, sensor readings, financial transactions, network traffic, social media posts, logs, and recommendation interactions (PDF pp. 620-622).

Traditional mining assumes stored data can be scanned multiple times. Stream mining cannot assume that.

174

CS 12 - Data Stream Mining: Stream Mining Constraints

Important constraints:

- Data may arrive continuously and quickly.
- Full storage may be impossible.
- Algorithms often get only one pass.
- Memory is limited.
- Processing time per item is limited.
- Data distribution may change over time.
- Results may need to be updated online.

The one-pass constraint is central: the algorithm must process each item as it arrives and maintain only summaries or model updates.

175

CS 12 - Data Stream Mining: Stationarity and Non-Stationarity

Stationary data has a stable distribution over time.

Non-stationary data changes distribution. In real streams, non-stationarity is common.

Examples:

- User interests change.
- Fraud patterns change.
- Network traffic changes during attacks.
- Sensor behavior changes due to seasons or faults.

176

CS 12 - Data Stream Mining: Concept Drift

Concept drift occurs when the relationship between input features and target output changes over time. A model trained on old data may become inaccurate.

Types:

- Sudden drift: abrupt change.
- Gradual drift: new concept slowly replaces old.
- Incremental drift: continuous movement.
- Recurring drift: old patterns return, such as seasonal behavior.

Handling drift requires adaptive models, sliding windows, forgetting factors, change detection, or retraining.

177

CS 12 - Data Stream Mining: Feature Relevance and Independence

The PDF discusses feature irrelevance using independence (PDF p. 623).

Formula - irrelevance/independence idea:

```
```text
P(X, Y) = P(X)P(Y)
```
```

If feature X is independent of target Y, it provides no predictive information about Y.

The PDF also distinguishes strong relevance and weak relevance in non-stationary analytics (PDF p. 623). A strongly relevant feature is essential for prediction; a weakly relevant feature may help only in combination with other features or under certain contexts. Stream mining must keep feature selection parsimonious because memory and update time are limited.

178

CS 12 - Data Stream Mining: Stream Sampling

Sampling selects a subset of stream items for analysis.

Reservoir sampling: maintains a uniform random sample of fixed size from a stream of unknown length. When the n th item arrives, it is accepted into the reservoir with probability related to reservoir size and n , replacing an existing item if accepted.

Use when full stream storage is impossible but representative samples are needed.

179

CS 12 - Data Stream Mining: Bloom Filters

A Bloom filter is a memory-efficient probabilistic data structure for set membership. It can answer:

- "Definitely not in set."
- "Possibly in set."

It can have false positives but no false negatives, assuming no deletion in the basic version.

Use cases:

- Duplicate detection.
- Membership checks.
- Stream filtering.

180

CS 12 - Data Stream Mining: Sketching

Sketching maintains compact approximate summaries of streams. It trades exactness for memory and speed.

Examples:

- Frequency estimation.
- Heavy hitters.
- Distinct counts.
- Approximate quantiles.

Sketching is useful when exact counts are too expensive.

181

CS 12 - Data Stream Mining: Synopsis Data Structures

A synopsis is a compact summary of data. Stream mining relies on synopses such as samples, histograms, wavelets, sketches, micro-clusters, and model summaries (PDF p. 642).

The point is to answer queries or update models without storing full data.

182

CS 12 - Data Stream Mining: Sliding Windows

A sliding window keeps recent data and discards older data. This is useful when recent behavior is more relevant than old behavior.

Types:

- Landmark window: from a fixed start time to now.
- Sliding window: most recent N records or most recent time interval.
- Damped window: older records have decreasing weight.

183

CS 12 - Data Stream Mining: Motifs and Stream Patterns

Motifs are repeated patterns in sequence/time-series streams. Detecting motifs helps summarize recurring behavior, while deviations from motifs may indicate anomalies.

184

CS 12 - Data Stream Mining: DSMS and CEP (1/2)

Data Stream Management Systems (DSMS) support continuous queries over streams. Complex Event Processing (CEP) detects meaningful event patterns from primitive events.

Examples:

- Detect fraud from transaction sequences.
- Detect network intrusion from packet/event patterns.
- Detect operational incidents from logs and metrics.

The PDF also frames stream systems through client/server, mobile-agent, and AI-agent settings, with storage, computation, and communication constraints (PDF pp. 633-636). It separates data-based techniques from task-based techniques:

- Data-based solutions: sample a subset, transform the stream to an approximate representation, or maintain sketches/synopses.
- Task-based solutions: adapt the mining algorithm itself using learning theory, complexity theory, approximation algorithms, anytime algorithms, and resource-aware updates (PDF pp. 641-643).

Research issues include continuous flow, device energy management, memory requirements, accuracy, robustness, model compression/selection, temporal data mining, interactive mining,

185

CS 12 - Data Stream Mining: DSMS and CEP (2/2)

ubiquitous computing, and granular computing (PDF p. 644).

186

CS 12 - Data Stream Mining: Task-Based Stream Mining

The same data mining tasks are adapted for streams:

- Stream classification.
- Stream clustering.
- Stream frequent pattern mining.
- Stream anomaly detection.
- Stream recommendation.

Each needs incremental updates and mechanisms for concept drift.

187

CS 12 - Data Stream Mining: Exam Pointers

- Stream mining differs from batch mining because data is unbounded and one-pass.
- Concept drift is a change in the target relationship over time.
- Reservoir sampling keeps a fixed-size random sample.
- Bloom filters allow false positives but not false negatives.
- Sketches/synopses are compact approximate summaries.

188

CS 12 - Data Stream Mining: Formulae/Numericals Highlight

- Formula: feature irrelevance under independence $P(X,Y)=P(X)P(Y)$ (PDF p. 623).
- Conceptual formula: one-pass stream algorithm = process current item + update compact state, without storing full stream (PDF pp. 620-649).
- Conceptual numerical: Bloom filter trades memory for false-positive probability; basic Bloom filter has no false negatives.

189

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining

Session divider

190

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Cybercrime Classification

The session covers cybercrime as an applied data mining problem. Cybercrime data may include network logs, device artifacts, communication records, social media, multimedia evidence, transactions, access logs, and graph relationships.

Possible mining tasks:

- Classification: classify activity as benign/malicious or case type.
- Clustering: group similar incidents, actors, or artifacts.
- Association mining: find co-occurring indicators.
- Anomaly detection: identify unusual behavior.
- Graph mining: analyze communication, transaction, or social networks.
- Stream mining: detect threats in real time.

The PDF frames cybercrime as activity where computers or networks can be the tool, the target, or the place of criminal activity. It lists cyber attacks, financial fraud, explicit content, human trafficking, dark internet activity, AI-driven cybersecurity, botnets, cross-border jurisdiction issues, and psychological/social harms such as depression, anxiety, PTSD, and trauma (PDF p. 650).

191

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Cybersecurity Framing and Kill Chain

Cybersecurity analysis uses the CIA triad: confidentiality, integrity, and availability. The PDF also connects AI model security/privacy with adversarial evaluation (PDF p. 650).

Attack and defense framing includes survey/assessment, exploitation, privilege escalation, maintaining access, denial of service, cyber kill chains, and TTPs. The PDF lists interruption, interception, modification, and fabrication as attack categories (PDF p. 651). It later gives an intrusion kill chain: reconnaissance, weaponization, delivery, exploitation, installation, command and control, and actions on objectives (PDF p. 663).

192

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Digital Forensics Workflow

Digital forensics focuses on collecting, preserving, analyzing, and presenting digital evidence.

A typical evidence pipeline:

1. Acquisition: collect data from devices, networks, cloud, applications, or media.
2. Preservation: maintain integrity and chain of custody.
3. Preprocessing: clean, deduplicate, normalize, extract metadata.
4. Feature extraction: derive useful attributes from text, images, logs, network records, or graph relations.
5. Mining/modeling: classification, clustering, anomaly detection, link analysis.
6. Interpretation: explain evidence and model outputs.
7. Reporting: present findings in legally/operationally useful form.

The PDF's digital-evidence steps are preservation, collection, validation, identification, analysis, interpretation, documentation, and presentation (PDF p. 652). It also lists computer/network forensics methodology as collection, examination, analysis, and reporting. Forensic analytics includes time analysis, metadata analysis, file analysis, application analysis, algorithm analysis, and mathematical analysis (PDF p. 653).

193

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Multimedia and Media Evidence

For media evidence, data mining may use:

- Image/video metadata.
- Visual features.
- Object/person/event detection.
- Temporal sequence analysis.
- Similarity search.
- Hashing/fingerprinting.
- Source/device attribution.

The PDF includes broader multimedia material later, but the CS 13 transcript emphasizes forensic/cyber use rather than detailed DFT/DWT/JPEG/MPEG mechanics.

194

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Security, Privacy, and Adversarial Issues

Security mining has additional constraints:

- Attackers adapt to detection models.
- Data can be intentionally manipulated.
- Labels may be delayed or incomplete.
- False positives can be costly.
- Privacy and legal constraints limit data access.
- Models need interpretability for investigation.

Adversarial robustness matters because malicious actors may change behavior to evade classifiers or anomaly detectors.

The PDF emphasizes practical media/cyber constraints: very large image/video volume, end-to-end encryption, varying media quality, blurred or low-resolution evidence, offenders adapting to evade detection, anonymous platforms, hidden networks, and lack of user awareness (PDF pp. 654-655).

195

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Cybercrime Solution Architecture (1/2)

The PDF's solution material includes distributed crawling/search and multi-model learning (PDF pp. 656-663):

- Apache Storm Crawler for recursive distributed web crawling.
- Elasticsearch for distributed search and analytics.
- Multi-model deep learning pipelines and learned tensor representations.
- YOLOv5-style object detection and Vision Transformer/foundation-model ideas such as multi-head self-attention and attention maps.
- Human-in-the-loop review for forensic robustness.
- Data pipeline: cleaning, cropping, augmentation, embedding, detection.
- Robustness criteria: low resolution, poor lighting, masks, ethnicity/generalization issues.
- Sensitivity analysis using false negative rate and false positive rate.
- Computational red teaming, AI-enabled adversaries, multi-stage campaigns, policy learning, parameter tuning, and portfolio construction.
- Robust optimization: distributional robustness, adversarial robustness, system robustness, black-box optimization, uncertainty estimates, and black-box versus white-box AI-system

196

**CS 13 - Cybercrime Case Study, Digital Forensics, and
Dynamic Graph Mining: Cybercrime Solution Architecture (2/2)**

threat modeling.

197

**CS 13 - Cybercrime Case Study, Digital Forensics, and
Dynamic Graph Mining: Graphs in Cyber and Social Data**

Many cyber problems are naturally graphs:

- Nodes: users, IPs, accounts, devices, domains, files, transactions.
- Edges: communication, access, transfer, friendship, shared attribute, co-occurrence.
- Weights: frequency, amount, duration, strength.
- Time: when edge/node activity occurs.

Graph mining helps reveal communities, dense components, central actors, suspicious structures, and evolving behavior.

198

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Dynamic Graphs and Graph Streams

Dynamic graphs change over time. Nodes and edges appear, disappear, or change weight. Examples include social networks, web graphs, communication networks, citation networks, and transaction networks (PDF pp. 686-757).

Questions:

- How does the graph grow?
- Are new dense communities forming?
- Are graph properties following expected laws?
- When do change points occur?
- Which nodes/edges are anomalous?

199

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Densification Power Law

Real graphs often densify: edges grow faster than nodes.

Formula - densification power law (PDF pp. 671, 737-740):

```
'''text
E(t) proportional to N(t)^alpha
'''
```

where:

- $E(t)$ = number of edges at time t .
- $N(t)$ = number of nodes at time t .
- $\alpha > 1$ indicates densification.

If $\alpha = 1$, edges grow linearly with nodes. If $\alpha > 1$, average degree increases over time.

200

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Shrinking Diameter

Many real graphs show shrinking effective diameter: as nodes and edges are added, average shortest paths can become shorter because new edges create shortcuts (PDF pp. 737-740).

This is counterintuitive but common in social/web graphs: the network grows but becomes more tightly connected.

201

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Small-World Properties

Small-world networks have short average path lengths and clustering/community structure. This helps explain why information can spread quickly in social, web, and communication networks.

202

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Dense Components

Dense-component mining finds subgraphs with unusually high internal connectivity. In cyber settings, dense components may indicate coordinated activity, communities, fraud rings, botnets, or operational groups.

203

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: k-Core, k-Clique, k-Plex, k-Club

Dense subgraph concepts:

- k-core: maximal subgraph where each node has degree at least k within the subgraph.
- k-clique: complete or near-complete highly connected group, depending on definition.
- k-plex: relaxed clique where each node may miss some edges.
- k-club: subgraph where distance between any two nodes is at most k .

These definitions capture different notions of density and closeness.

204

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Low-Rank Tensors and Multiway Data

Dynamic graphs can be represented as tensors: source node, destination node, time, relation type, or other dimensions. Low-rank tensor methods find latent patterns, communities, or anomalies across multiple modes.

Use cases:

- Communication patterns over time.
- User-item-time interactions.
- IP-domain-time events.
- Multi-relational social networks.

The PDF connects low-rank approximation with evolving user-item graphs, iterative algorithms, matrix factorization, least-square approximation error, alternating least squares optimization, and link prediction at $t+1$ using the network at time t (PDF p. 676).

205

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Graph Evolution and Change Points

Change-point detection identifies times when graph behavior shifts. Signals can include sudden changes in:

- Edge count.
- Degree distribution.
- Component size.
- Diameter.
- Triangle count.
- Community structure.
- Dense subgraph structure.
- Node/edge weights.

This connects anomaly detection with graph streams.

The PDF also describes evolutionary clustering and graph-stream maintenance: models should balance freshness at the current time step against consistency with historical clusters. Change points can be detected when the encoding cost or MDL description length of consecutive graph snapshots increases significantly (PDF pp. 672-675, 683).

206

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Exam Pointers

- Cybercrime mining is not one algorithm; it is a pipeline using multiple mining tasks.
- Digital forensics requires evidence integrity and interpretability.
- Cyber kill-chain stages and CIA triad are part of the security framing.
- Cybercrime solutions combine crawling/search, deep learning, tensor representations, red teaming, and human review.
- Graph representation is natural for cyber/social/transaction data.
- Densification means edges grow superlinearly with nodes.
- Shrinking diameter means graph gets more connected despite growth.
- Dense subgraph definitions differ; know k-core versus clique-style ideas.

207

CS 13 - Cybercrime Case Study, Digital Forensics, and Dynamic Graph Mining: Formulae/Numericals Highlight

- Formula: densification power law $E(t)$ proportional to $N(t)^\alpha$, with $\alpha > 1$ for densification (PDF pp. 671, 737-740).
- Conceptual formula: dynamic graph $G(t) = (V(t), E(t))$, where vertices and edges change over time.
- Conceptual numerical: shrinking diameter tracks shortest-path/effective-diameter trend over time (PDF pp. 737-740).
- Formula: false negative rate $FNR = FN / (TP + FN)$ and false positive rate $FPR = FP / (FP + TN)$ are used in cybercrime sensitivity analysis (PDF p. 659).
- Definition: k-core requires every node in the subgraph to have degree at least k within that subgraph.

208

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association

Session divider

209

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Are Real Graphs Random?

The session asks whether real graphs are random. The answer emphasized in the transcript is no: real graphs show patterns, laws, structure, communities, densification, shrinking diameters, power-law degree distributions, and anomalous deviations from expected graph behavior.

Random graph models are useful baselines, but real graphs usually deviate from pure randomness.

210

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Power Laws in Graphs

Many graph properties follow power-law-like behavior (PDF pp. 694-708):

- Degree distribution: few nodes have very high degree; many nodes have low degree.
- Weighted degree distribution.
- Eigenvalue/rank patterns.
- Triangle participation.
- Component sizes.

On log-log plots, power laws often appear approximately linear.

211

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Degree Distribution

Degree is number of edges incident to a node. In directed graphs:

- In-degree: incoming edges.
- Out-degree: outgoing edges.

In many real graphs, degree distribution is heavy-tailed: hubs exist.

212

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Weighted Graph Power Laws

In weighted graphs, edge weights may represent amount, frequency, duration, or strength. The transcript discusses donor/politician-style weighted graph examples where total incoming weight relates nonlinearly to in-degree.

Formula/relationship - weighted graph fortification (PDF p. 735):

```
```text
weight grows super-linearly with in-degree
 $1.01 < iw < 1.26$
```
```

Interpretation: nodes with more incoming links often receive disproportionately higher total weight.

213

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Triangle Law

Triangles measure local clustering and transitivity.

Formula - triangle count using eigenvalues (PDF pp. 694-708):

```
```text
#triangles = (1/6) * sum_i lambda_i^3
```
```

where λ_i are eigenvalues of the adjacency matrix. The factor $1/6$ accounts for repeated counting of each triangle.

214

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Diameter and Gelling Point

Graph diameter/effective diameter measures shortest-path spread.
The transcript explains that graph diameter may grow, shrink, or stabilize depending on graph evolution. Some networks show a spike and then stabilization; this stabilization/spike behavior is described as a gelling point in the lecture discussion.

The key mining question:

- What is the rate of graph growth or shrinkage?
- When does it stabilize?
- What law explains the observed trend?

215

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Logistic and Log-Logistic Fits

Graph trends over time can be fit using standard curve models, including sigmoid/logistic and log-logistic forms (PDF pp. 751-753).

Formula - sigmoid/logistic function:

```
'''text
sigmoid(x) = 1 / (1 + e^(-x))
'''
```

The transcript connects this to curves that grow and then stabilize.

216

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Graph Anomaly Detection

Graph anomalies are graph elements or substructures that deviate from expected patterns:

- Suspicious node.
- Suspicious edge.
- Dense subgraph.
- Sudden change in graph property.
- Abnormal egonet.
- Unusual weighted behavior.
- Synchronization anomaly.

Anomaly definition depends on context. In graph mining, "normal" may be expressed through degree laws, weight-degree relations, egonet relations, temporal trends, or fitted models.

217

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: OddBall

OddBall detects anomalous nodes by examining egonet features (PDF pp. 760-762). An egonet is a node, its neighbors, and edges among them.

Features include:

- Egonet degree.
- Number of egonet edges.
- Total egonet weight.
- Principal eigenvalue.

OddBall looks for nodes whose egonet feature relationships deviate from expected power-law patterns.

218

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: CatchSync and Synchronization Anomalies

CatchSync-style graph anomaly detection looks for synchronized behavior. In many applications, suspicious accounts/nodes act together in unusually coordinated ways. Examples include coordinated fraud, bot behavior, review spam, or repeated simultaneous actions.

The mining idea is to detect groups whose temporal or relational behavior is too synchronized compared with normal patterns.

219

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Graph Generators

Graph generators create synthetic graphs for simulation, benchmarking, and understanding which mechanisms produce observed graph properties.

220

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Erdos-Renyi Random Graphs

In an Erdos-Renyi graph, edges are generated randomly, often with independent probability p . This is a baseline random graph model.

Limitation: it does not naturally reproduce many real graph properties such as heavy-tailed degree distributions, communities, and densification.

221

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: R-MAT

R-MAT recursively partitions the adjacency matrix and places edges according to probabilities. It can generate graphs with skewed degree distributions and community-like structure.

222

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Kronecker Graphs

Kronecker graphs use recursive expansion from a small initiator matrix (PDF pp. 779-787). They are useful because self-similarity can produce several real graph properties:

- Power-law degree distributions.
- Small or shrinking diameter.
- Communities within communities.
- Densification-like behavior.

223

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Graph Partitioning

Graph partitioning divides a graph into parts while minimizing cross-part edges and balancing partition sizes (PDF pp. 797-799). It is useful for parallel processing, clustering, community detection, and graph compression.

Good partitioning wants:

- Many edges within partitions.
- Few edges across partitions.
- Balanced partition sizes.

224

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: METIS

METIS is a multilevel graph partitioning approach (PDF pp. 797-798):

1. Coarsen graph: collapse nodes/edges to smaller graph.
2. Partition smaller graph.
3. Uncoarsen/refine partition back to original graph.

This is efficient for large graphs and often produces high-quality partitions.

225

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Spectral Partitioning

Spectral partitioning uses eigenvectors of a graph matrix such as the Laplacian.

Formula - graph Laplacian (PDF pp. 912-914):

```
```text
L = D - W
```
```

where D is degree matrix and W is weighted adjacency matrix. Eigenvectors provide a low-dimensional representation useful for partitioning or clustering.

226

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Co- Clustering and Cross-Association

Co-clustering simultaneously clusters rows and columns of a matrix. Cross-association applies this idea to binary matrices/graphs and finds row groups and column groups that form homogeneous blocks (PDF pp. 805-807).

Example:

- Rows: users.
- Columns: items.
- Matrix entries: interactions.
- Cross-association discovers user groups and item groups together.

227

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: MDL in Cross-Association

Cross-association uses Minimum Description Length to automatically choose groupings.

Formula - MDL idea:

```
```text
best grouping = grouping with minimum total encoding cost
```
```

Total encoding cost includes the cost of describing the model/block structure and the cost of describing deviations/errors.

228

**CS 14 - Graph Pattern Mining, Graph Anomalies,
Generators, Partitioning, and Cross-Association: Multimedia/DSP Appendix Topics From the PDF**

The consolidated PDF also contains multimedia/mining material around PDF pp. 822-884. The CS 14 transcript mainly emphasizes graph pattern mining, but these PDF pages are included here so the consolidated notes do not skip them.

229

**CS 14 - Graph Pattern Mining, Graph Anomalies,
Generators, Partitioning, and Cross-Association: DFT and DWT for Signal Mining**

The multimedia section frames a signal as a sequence over time or space, such as sales over time, packets per day, virus infections per month, BGP updates, audio, or image intensity (PDF pp. 822-825). The mining goals are to find patterns, periodicities, similarity, anomalies, forecasts, or compressed representations.

The PDF contrasts:

- DFT/Fourier: good for spotting periodicities and global frequencies.
- DWT/Wavelets: good for multi-resolution analysis, spikes, short-duration events, and compression (PDF pp. 823-829).

DFT performs poorly for isolated spikes because many coefficients may be needed. Short-window Fourier transform helps with localized events but requires choosing a window size. DWT handles multiple window sizes and maps coefficients into a time-frequency view (PDF pp. 827-829).

230

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Haar Wavelets

Haar wavelets recursively compute smooth and difference components (PDF pp. 830-834). At each level, adjacent values are combined into averages/smooth components and differences/high-frequency components.

Formula - Haar smooth and difference (PDF p. 834):

```
...text
diff[i] = (vals[2i] - vals[2i+1]) / sqrt(2)
smooth[i] = (vals[2i] + vals[2i+1]) / sqrt(2)
...
```

Then the same process repeats on the smooth vector. The input length is typically a power of 2 for the simple Haar transform.

Wavelet families include Haar, Daubechies, Coifman, Morlet, and Gabor (PDF p. 834).

231

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: DWT Uses and Advantages

DWT is useful for spikes, chirps, mixed periodicities, BGP routing update anomalies, image compression, and multi-dimensional data (PDF pp. 836-855). Advantages listed in the PDF:

- Better compression for many signals.
- Handles spikes well.
- Supports progressive transmission.
- Often fast to compute, $O(n)$.
- Gives multi-resolution representation similar to how visual/audio perception uses local structure.

Exam shortcut: DFT -> periodicities; DWT -> localized/multi-resolution events and compression (PDF p. 855).

232

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: JPEG Compression

JPEG is for image compression and can be lossy or lossless, grayscale or color (PDF pp. 858-865).

Lossy grayscale JPEG outline (PDF p. 862):

1. Split image into 8x8 blocks.
2. Apply DCT to each block.
3. Quantize coefficients, which reduces precision and creates loss.
4. Encode coefficients:
 - DC coefficient is delta-encoded from neighbors.
 - AC coefficients are read in zig-zag order and Huffman encoded.

Numerical: the PDF states a typical lossy JPEG result of about 0.75-1.5 bits/pixel, roughly 8:1 compression, with sufficient quality for many applications (PDF p. 862).

Lossless JPEG uses predictive coding and encodes prediction errors; the PDF notes typical compression around 2:1 (PDF p. 863).

Color JPEG treats image components as channels/bands, such as red/green/blue or spectral bands, and may interleave components to pipeline decompression/display (PDF pp. 863-865).

233

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: MPEG Compression

MPEG is for video compression. A video is many still images, but applying JPEG independently to every frame wastes redundancy because consecutive frames are often similar (PDF pp. 866-869).

MPEG balances compression and random access:

- I-frames: intra-coded frames, compressed like still images; support random access.
- P-frames: predicted from earlier I/P frames.
- B-frames: bidirectionally/interpolated frames; not used as reference in the PDF description.
- Motion fields/motion estimation capture movement between frames.

Exam point: MPEG improves over frame-by-frame JPEG by using inter-frame similarity and motion prediction, while keeping I-frames for access and error recovery.

234

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Fractal Compression and IFS (1/2)

Fractal compression uses self-similarity. The PDF uses examples such as the Sierpinski triangle, Koch snowflake, fern leaf, and self-similar traffic (PDF pp. 870-883).

The core idea is Iterated Function Systems (IFS): describe an image or shape using a small set of transformations whose repeated application recreates the object.

Formula - 2D affine transformation (PDF p. 871):

```
...text
x' = a x + b y + e
y' = c x + d y + f
...
```

There are 6 coefficients: a,b,c,d for rotation/scaling/shear-like matrix behavior, and e,f for translation. For the Sierpinski triangle, the PDF shows three such transformations with probabilities (PDF pp. 871-873).

Decompression can be:

- Iterative deterministic application of transformations, but this can explode as 3^k sub-images after k steps for three transformations (PDF p. 874).

235

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Fractal Compression and IFS (2/2)

repeatedly choose a transformation according to probabilities, generate the next point, and ignore early burn-in points (PDF p. 875).

Fractal compression can be lossy, can support zooming without JPEG-style blockiness, and can model self-similar traffic such as 80-20 style distributions (PDF pp. 880-883).

236

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Exam Pointers

- Real graphs are not purely random; they show recurring laws.
- Power laws often appear linear on log-log plots.
- Densification and shrinking diameter are key graph evolution laws.
- OddBall uses egonet feature deviations.
- Kronecker graphs model self-similarity and real graph properties.
- METIS uses coarsen-partition-uncoarsen.
- Cross-association co-clusters rows and columns using MDL.
- Multimedia PDF appendix: DFT spots periodicity; DWT handles localized/multi-resolution events; JPEG uses block-DCT/quantization; MPEG uses I/P/B frames and motion; fractal compression uses affine self-similar transformations.

237

CS 14 - Graph Pattern Mining, Graph Anomalies, Generators, Partitioning, and Cross-Association: Formulae/Numericals Highlight

- Formula: densification $E(t)$ proportional to $N(t)^\alpha$, $\alpha > 1$ (PDF pp. 737-740).
- Formula: triangle count $\#triangles = (1/6) \sum_i \lambda_i^3$ (PDF pp. 694-708).
- Formula: weighted graph fortification exponent $1.01 < i_w < 1.26$ (PDF p. 735).
- Formula: sigmoid/logistic $1/(1+e^{-x})$ for trend fitting (PDF pp. 751-753).
- Formula: graph Laplacian $L=D-W$ for spectral clustering/partitioning (PDF pp. 912-914).
- Conceptual formula: Cross-association/MDL minimizes total encoding cost (PDF pp. 805-807).
- Formula: Haar diff[i] = $(vals[2i] - vals[2i+1]) / \sqrt{2}$, smooth[i] = $(vals[2i] + vals[2i+1]) / \sqrt{2}$ (PDF p. 834).
- Numerical: JPEG lossy compression around 0.75-1.5 bits/pixel, roughly 8:1; lossless prediction around 2:1 (PDF pp. 862-863).
- Formula: fractal/IFS affine transform $x' = ax + by + e$, $y' = cx + dy + f$; 6 coefficients (PDF p. 871).
- Numerical: naive deterministic decompression with 3 transformations grows as 3^k sub-images after k steps (PDF p. 874).

238

Consolidated Formula and Numerical Revision Sheet

239

Data Similarity and Information

- Euclidean distance: $d(x,y)=\sqrt{\sum_k (x_k-y_k)^2}$ (PDF p. 31).
- Minkowski distance: $d_r(x,y)=(\sum_k |x_k-y_k|^r)^{1/r}$;
Manhattan is $r=1$, Euclidean is $r=2$, supremum is $r \rightarrow \infty$
(PDF pp. 32-33).
- SMC: $(f_{11}+f_{00})/(f_{01}+f_{10}+f_{11}+f_{00})$ (PDF p. 36).
- Jaccard: $f_{11}/(f_{01}+f_{10}+f_{11})$ (PDF p. 36).
- Entropy: $H(X)=-\sum_i p_i \log_2(p_i)$; fair coin entropy is 1 bit
(PDF pp. 41-42).
- Mutual information: $I(X,Y)=H(X)+H(Y)-H(X,Y)$ (PDF p. 43).

240

Decision Trees

- Gini: $Gini(t) = 1 - \sum_i p_i(t)^2$ (PDF pp. 73-76).
- Weighted Gini split: $Gini_split = \sum_i (n_i/n) Gini(i)$ (PDF p. 76).
- Entropy: $Entropy(t) = -\sum_i p_i(t) \log_2 p_i(t)$ (PDF pp. 78-82).
- Information gain:
 $Gain = Entropy(parent) - \text{weighted_entropy}(children)$ (PDF pp. 78-82).
- Gain ratio: $Gain\ Ratio = Gain_split / SplitInfo$;
 $SplitInfo = -\sum_i (n_i/n) \log_2 (n_i/n)$ (PDF pp. 82-83).
- Classification error: $Error(t) = 1 - \max_i p_i(t)$ (PDF p. 83).
- MDL: $Cost(Model, Data) = Cost(Data|Model) + x * Cost(Model)$ (PDF pp. 97-100).
- Pessimistic pruning numerical: before $(10 + 0.5)/30 = 10.5/30$, after $(9 + 4 * 0.5)/30 = 11/30$ (PDF p. 101).

241

Neural Networks and Ensembles

- Perceptron: $Y = \text{sign}(w_0 + \sum_i w_i x_i)$; example
 $Y = \text{sign}(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$ (PDF p. 106).
- Gradient update: $w \leftarrow w - \eta * dLoss/dw$ (PDF pp. 106-113).
- Ensemble condition: base classifiers should be better than random guessing, i.e., binary error rate below 0.5 (PDF p. 116).
- AdaBoost numerical: weight updates and final sign vote shown on PDF p. 127.

242

Classification Metrics

- Accuracy: $(TP+TN)/(TP+TN+FP+FN)$ (PDF p. 130).
- Precision: $TP/(TP+FP)$ (PDF pp. 130-140).
- Recall/TPR: $TP/(TP+FN)$ (PDF pp. 130-140).
- F1: $2pr/(p+r)$ (PDF pp. 130-140).
- FPR: $FP/(FP+TN)$ (PDF pp. 136-140).
- AUC: ideal 1.0; random 0.5 (PDF pp. 136-140).

243

Association and Frequent Patterns

- Support count: $\sigma(X)=\text{count}(\text{transactions containing } X)$ (PDF pp. 165-167).
- Support: $\text{support}(X)=\sigma(X)/N$ (PDF pp. 165-167).
- Confidence: $\text{confidence}(X \rightarrow Y)=\text{support}(X \cup Y)/\text{support}(X)$ (PDF pp. 165-167).
- Apriori: infrequent itemset $\rightarrow$ all supersets infrequent (PDF pp. 168-184).
- Lift: $P(X,Y)/(P(X)P(Y))=P(Y|X)/P(Y)$ (PDF pp. 200-203).
- Tea/Coffee numerical: $\text{confidence}=.75$, $P(\text{Coffee})=.8$, $\text{lift}=.9375$; negative association (PDF pp. 200-203).
- Cross-support ratio: $r(X)=\min_i s(x_i)/\max_i s(x_i)$ (PDF p. 212).
- h-confidence: $\text{hconf}(X)=s(X)/\max_i s(x_i)$ and $\text{hconf}(X) \leq r(X)$ (PDF pp. 213-214).
- Z-test for association: $Z=(\mu-\mu'-\Delta)/\sqrt{s_1^2/n_1+s_2^2/n_2}$; example $Z=3.11 > 1.64$ (PDF pp. 224-225).

244

Clustering

- K-means SSE: $SSE = \sum_i \sum_{x \in C_i} \text{dist}(x, m_i)^2$ (PDF p. 271).
- Cluster validity numerical: $k=1$: $SSE=10$; $k=2$: $SSE=1$, $SSB=9$, total 10 (PDF p. 302).
- Silhouette: $s=(b-a)/\max(a,b)$, range $[-1,1]$ (PDF p. 303).
- DBSCAN: parameters Eps, MinPts; core/border/noise points (PDF pp. 296-299).
- Spectral clustering: $L=D-W$ (PDF pp. 912-914).

245

Anomaly and Statistical Testing

- Grubbs statistic: $G = \max |X - \bar{X}|/s$ (PDF p. 320).
- Density estimate: $\text{density}(x) = 1/\text{dist}(x, k)$ (PDF p. 331).
- Reconstruction error: $\|x - \hat{x}\|$ (PDF p. 340).
- FWER: $1 - (1 - \alpha)^m$; for $\alpha = .05$, $m = 10$, FWER is about .401. PDF p. 943 prints .60, but that is approximately the no-error probability $.95^{10}$.
- Bonferroni: $\alpha^* = \alpha/m$ (PDF p. 943).
- FDR quantity: $Q = FP/(TP + FP)$ when positives are reported (PDF pp. 944-945).

246

Recommenders, Streams, and Graphs

- Collaborative filtering model: $\text{Model} = f(\text{utility matrix})$ (PDF p. 506).
- Frequent-pattern recommender score: $\text{score}(\text{rule } k) = \text{support}(\text{rule } k) * \text{confidence}(\text{rule } k)$ (PDF p. 546).
- Feature independence: $P(X, Y) = P(X)P(Y)$ (PDF p. 623).
- Densification power law: $E(t)$ proportional to $N(t)^\alpha$, $\alpha > 1$ (PDF pp. 671, 737-740).
- Triangle count: $\# \text{triangles} = (1/6) \sum_i \text{lambda}_i^3$ (PDF pp. 694-708).
- Weighted graph exponent: $1.01 < i_w < 1.26$ (PDF p. 735).
- Sigmoid: $1/(1+e^{-x})$ (PDF pp. 751-753).
- Graph Laplacian: $L = D - W$ (PDF pp. 912-914).
- Cross-association: choose row/column grouping that minimizes MDL encoding cost (PDF pp. 805-807).

247

Multimedia/DSP Appendix

- Haar DWT: $\text{diff}[i] = (\text{vals}[2i] - \text{vals}[2i+1]) / \sqrt{2}$ and $\text{smooth}[i] = (\text{vals}[2i] + \text{vals}[2i+1]) / \sqrt{2}$ (PDF p. 834).
- DFT vs DWT: DFT spots global periodicities; DWT handles localized spikes, short-duration events, and multi-resolution compression (PDF pp. 827-855).
- JPEG numerical: lossy JPEG around 0.75-1.5 bits/pixel, roughly 8:1; lossless predictive JPEG around 2:1 (PDF pp. 862-863).
- MPEG structure: I-frames for intra-coded/random access, P-frames for prediction, B-frames for interpolation (PDF pp. 866-869).
- Fractal affine transform: $x' = ax + by + e$, $y' = cx + dy + f$, 6 coefficients (PDF p. 871).
- Fractal numerical: deterministic decompression with three transformations can grow as 3^k sub-images after k steps (PDF p. 874).

248

Final Revision Strategy

1. First revise CS 1-2 to lock terminology: task types, attributes, distance, preprocessing.
2. Then revise CS 3-6 as the classification block: decision trees, overfitting, ANN, ensembles, imbalanced metrics, KNN, rules.
3. Then revise CS 6-7 as the association block: support, confidence, Apriori, lift, closed/maximal, support skew.
4. Then revise CS 8-9 as unsupervised/anomaly block: clustering algorithms, validation, anomaly detection, multiple testing.
5. Then revise CS 10-14 as applications/advanced mining: recommenders, text, streams, cyber, graphs, and the multimedia/DSP appendix pages.
6. Memorize formulas from the revision sheet and practice numerical substitutions for entropy/Gini/support-confidence/lift/SSE/silhouette/confusion-matrix metrics/FWER/Haar wavelet/JPEG-MPEG-fractal basics.